Hands-On Lab

Create Task Definitions, Scheduling Tasks, Configuring Services, and Cluster using Fargate

Scenario:

In an e-commerce platform, a microservices architecture is utilized for better scalability and maintenance. Using AWS Fargate, individual services like user authentication, product catalog, and order processing are containerized. Task definitions are created for each service. These tasks are scheduled to ensure availability during peak shopping hours. The services are configured in a Fargate cluster, optimizing resource allocation and cost, while ensuring high availability and load balancing. This setup allows for efficient handling of varying traffic loads, enhancing user experience and system reliability.

Description:

In this Activity Guide, we first create an ECS cluster as a logical grouping for our services. Then, we define tasks, specifying container images and configurations. Finally, we create a service within the ECS cluster, linking it to the task definitions. This service manages task scheduling and scaling, ensuring efficient, serverless container management.

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

K21Academy
Learn Cloud, Data & AI from Experts

Contents

1 Introduction **3**
2 Overview & Key Concepts **5**
2.1 AWS Fargate 5
2.2 Task Definitions 5
2.3 Scheduling Tasks 5
2.4 Configuring Services 6
2.5 Cluster Management 6
3 Documentation Links **7**
4 Prerequisites **8**
5 Cost to Perform this Lab **9**
6 Security groups **10**
6.1 Creating Security Groups 10
7 Creating ECS Cluster **14**
8 Creating task definitions **17**
9 Create a service **22**
10 Share your learnings **27**
10.1 Share Hands-On Lab on LinkedIn to Build Authority & Job Opportunities 28
10.2 Share win in the Community 31
11 Deleting/Cleanup **34**
11.1 Deregister the Tasks 34
11.2 Deleting the Cluster 35
12 Troubleshooting **38**
13 Summary **42**

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

1 INTRODUCTION

Amazon Elastic Container Service (ECS) is a fully managed container orchestration service that helps you to more efficiently deploy, manage, and scale containerized applications.

It deeply integrates with the AWS environment to provide an easy-to-use solution for running container workloads in the cloud and on-premises with advanced security features using Amazon ECS Anywhere.

When you create an Amazon ECS service, you specify the basic parameters that define what makes up your service and how it should behave.

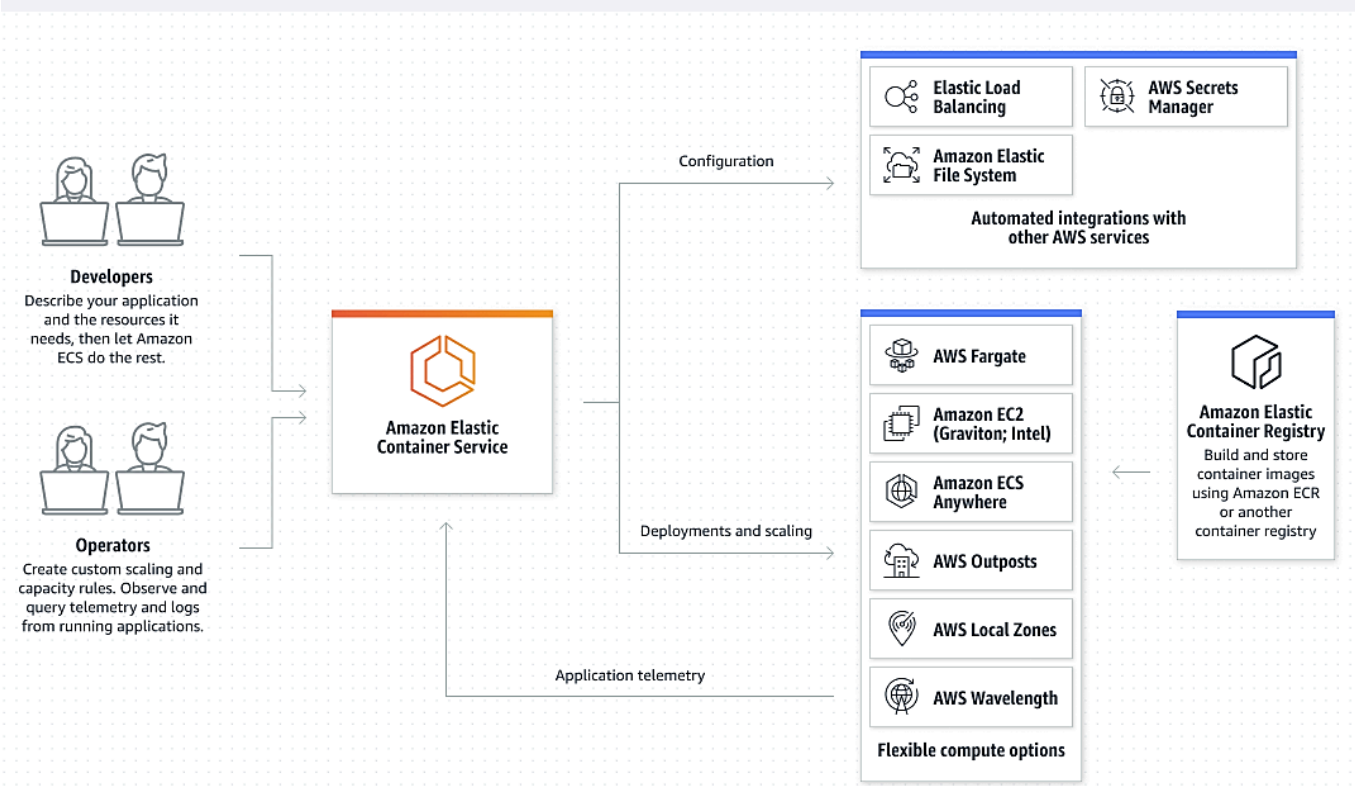
These parameters create a service definition.

AWS Fargate is a serverless, pay-as-you-go compute engine that lets you focus on building applications without managing servers.

Moving tasks such as server management, resource allocation, and scaling to AWS does not only improve your operational posture, but also accelerates the process of going from idea to production on the cloud and lowers the total cost of ownership.

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.



This activity guide covers steps for:

1. Create ECS Cluster
2. Create Task Definition
3. Create a Service
4. Delete Resources

K21Aca

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

2 Overview & Key Concepts

In this section, we'll cover key concepts related to AWS Fargate, including an

2.1 AWS Fargate

AWS Fargate is a serverless compute engine for containers that runs applications without requiring users to manage the underlying infrastructure. It works seamlessly with Amazon ECS and EKS, allowing users to focus on deploying and managing containerized applications.

Fargate eliminates the need to provision, configure, or scale EC2 instances, offering a fully managed environment for running containers efficiently.

2.2 Task Definitions

A Task Definition is a template that specifies how Docker containers should run within Amazon ECS.

Key components of a task definition include:

- Container image
- CPU and memory settings
- Networking mode
- IAM roles and permissions
- Logging and storage configurations

Task definitions allow users to create repeatable configurations for running container workloads.

2.3 Scheduling Tasks

ECS offers multiple scheduling options to control when and how tasks run:

- **Service Scheduler** – Ensures that tasks run continuously.
- **Run Task API** – Runs a task on demand.

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

- **Scheduled Tasks** – Uses EventBridge (formerly CloudWatch Events) to run tasks at specific intervals (like cron jobs).
- **Capacity Providers** – Automatically adjust resource allocation for cost optimization.

2.4 Configuring Services

Amazon ECS Services ensure high availability and scalability of containerized applications.

Key configurations include:

- Defining task replicas
- Attaching Application Load Balancers (ALB) or Network Load Balancers (NLB)
- Enabling automatic scaling based on CPU/memory usage
- Updating deployments with minimal downtime

2.5 Cluster Management

AWS Fargate abstracts cluster management, automatically handling the scaling and provisioning of compute resources. Users do not need to manage EC2 instances, making Fargate an efficient and cost-effective solution for containerized applications.

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

3 Documentation Links

1. Amazon ECS
   https://aws.amazon.com/ecs/

2. AWS ECS Cluster
   https://docs.aws.amazon.com/AmazonECS/latest/developerguide/clusters.html

3. AWS ECS Features
   https://aws.amazon.com/ecs/features/

4. AWS ECS Task Definitions
   https://docs.aws.amazon.com/AmazonECS/latest/developerguide/task_definitions.html

5. How to Create a Service in AWS ECS?
   https://docs.aws.amazon.com/AmazonECS/latest/developerguide/create-service.html

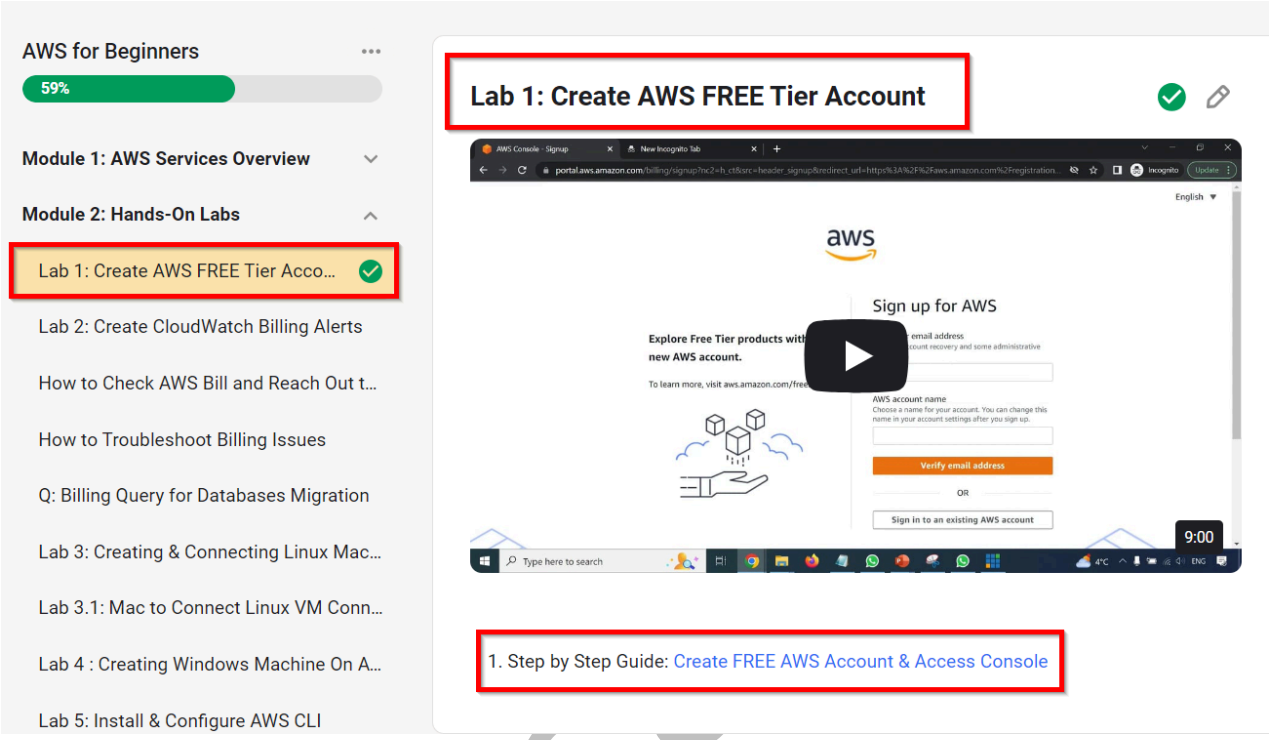
6. AWS Fargate
   https://aws.amazon.com/fargate/

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

4 PREREQUISITES

1. You must have an AWS Account. [Check out the Step-by-step Guide Here]



K21Ac

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

5 Cost to Perform this Lab

Total Estimated Cost for 3-5 Hours:

- **For 3 hours:** Approx. **$0.54**
- **For 5 hours:** Approx. **$0.90**

Estimated Costs for 3-5 Hours of the Lab:

1. AWS Fargate:
   - Task Execution: 0.25 vCPU & 0.5 GB memory.
   - 3 hours: Approx. $0.54
   - 5 hours: Approx. $0.90

2. Amazon ECS:
   - ECS Cluster Management: Free

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

6 SECURITY GROUPS

A Security Group acts as a virtual firewall for your Amazon EC2 instances, controlling inbound and outbound traffic based on defined rules.

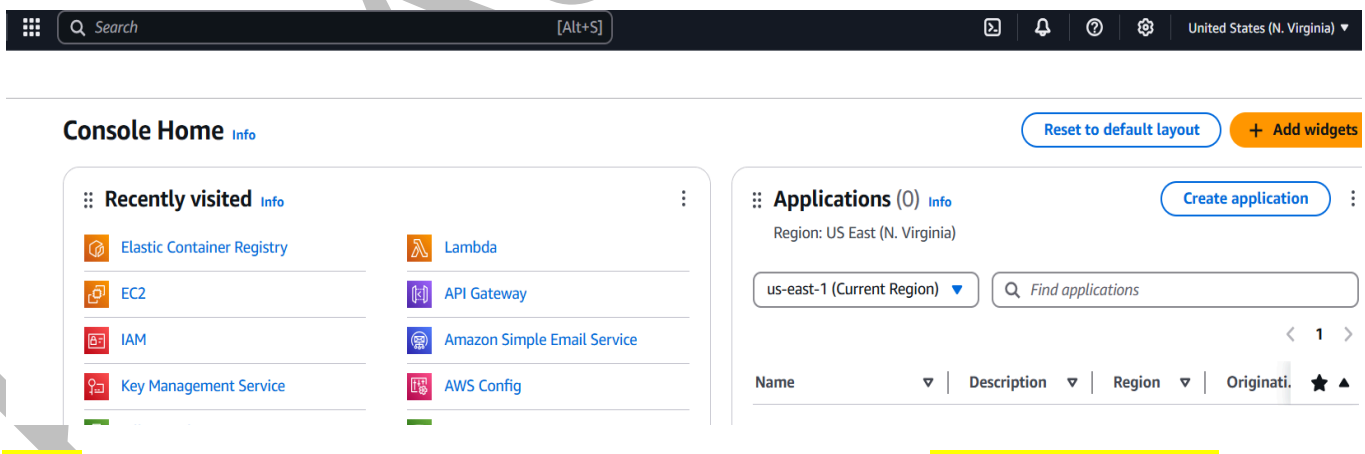
AWS allows users to configure Security Groups to restrict access to instances, enhancing security by permitting only necessary traffic.

Security Groups are stateful, meaning that if an inbound rule allows traffic, the corresponding outbound response is automatically permitted. Users can define rules based on IP addresses, ports, and protocols to customize access control.

6.1 Creating Security Groups

Note: In this section, we will be providing detailed steps for creating a security group in EC2. For this lab, we are using the **N. Virginia Region**. You can select any region that suits your preferences.

1. Log in to your **AWS Management Console**.

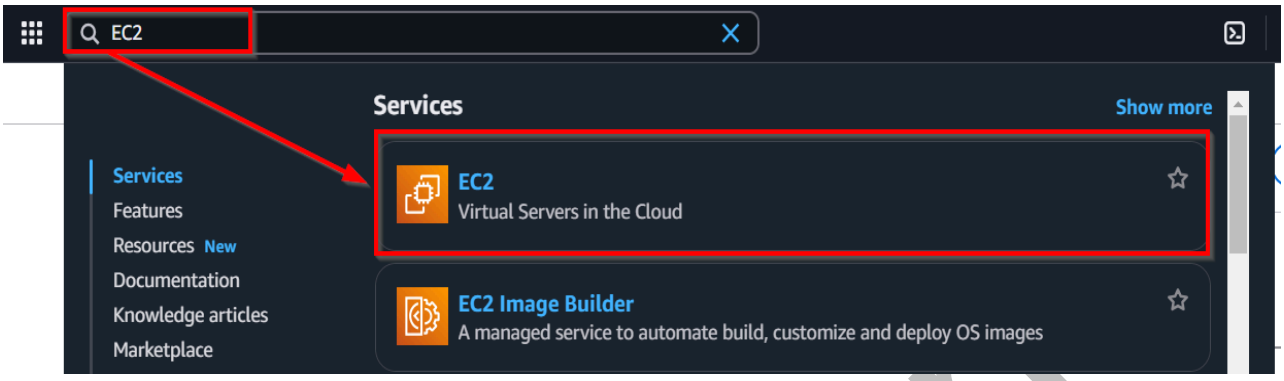


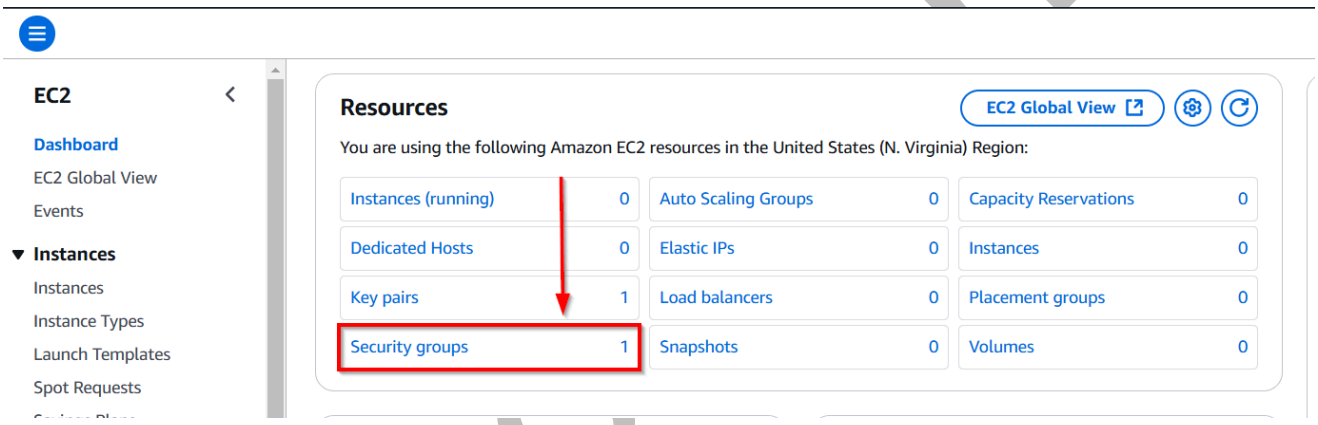
Note: Here we are using the AWS Fargate Service, which is a chargeable service. So, we suggest you please delete all the resources once you have completed the Activity Guide.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

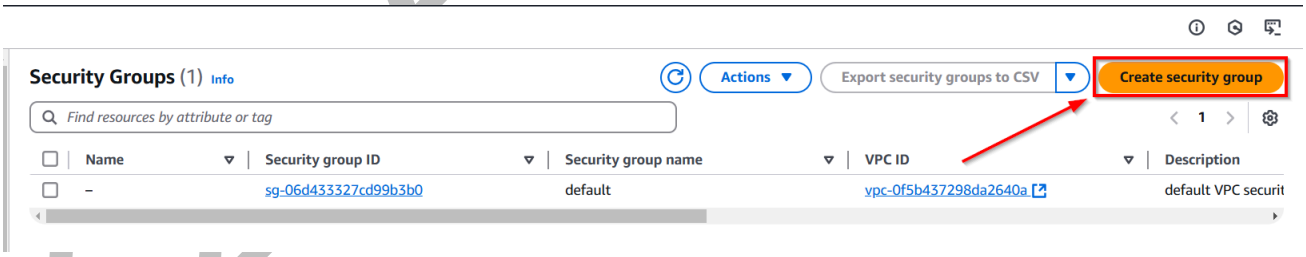
2. Follow the below steps to create a Security Group.

   In the **EC2 Console**, click on **Security Groups** in the left **Navigation Panel.**





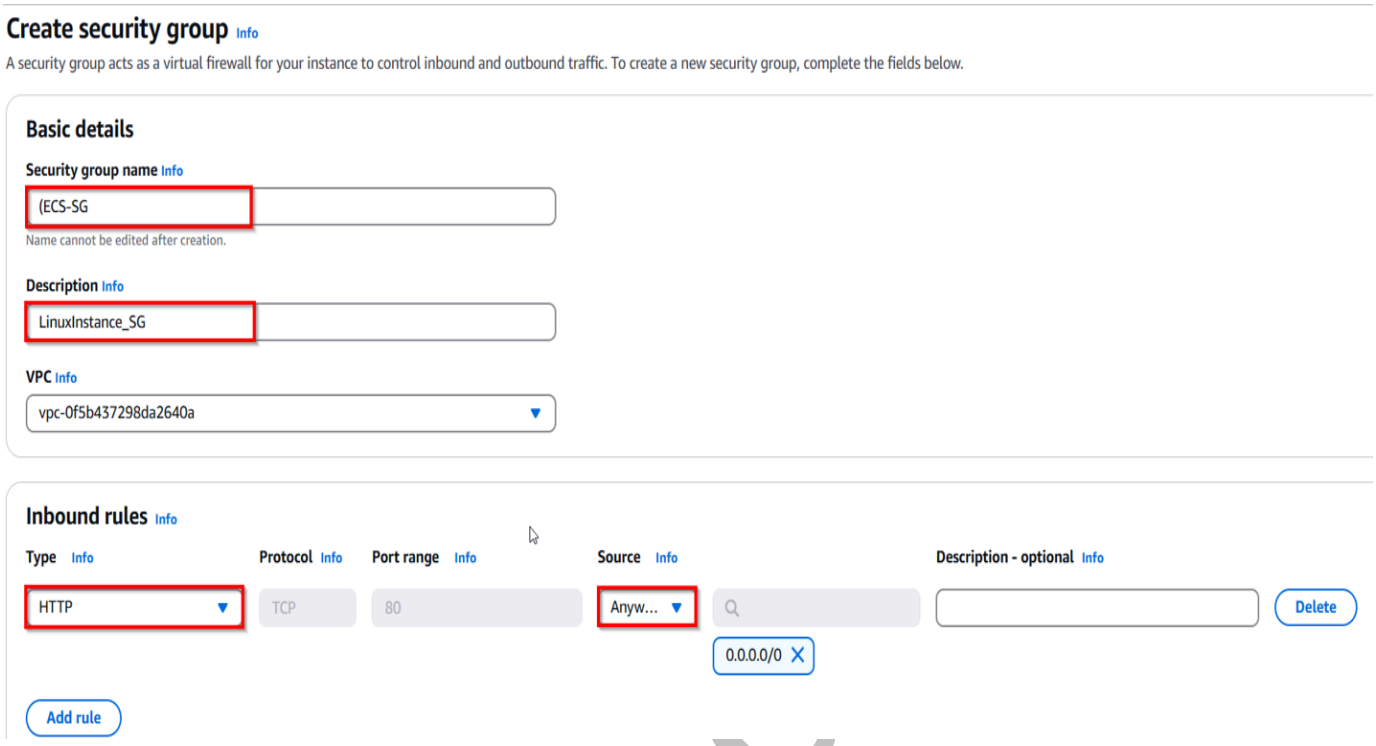
3. Click on the security groups



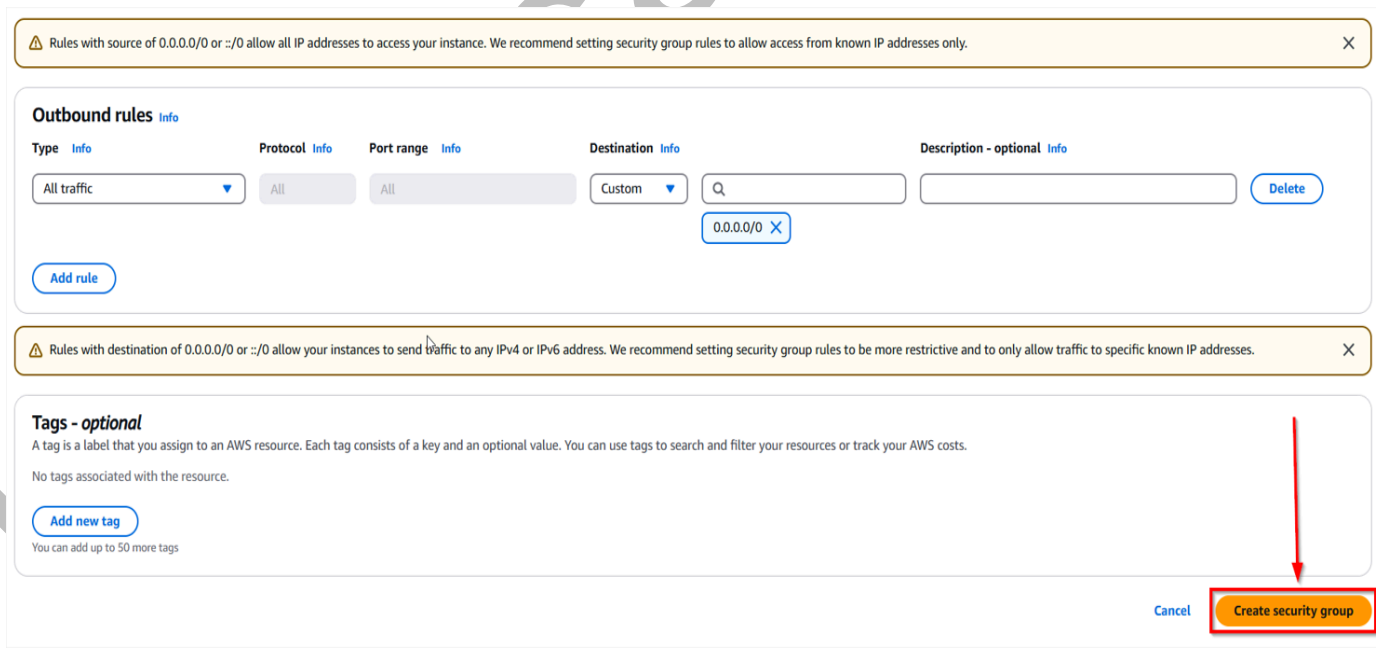
4. Give the **name (ECS-SG)** and **description (LinuxInstance_SG)** of the **Security Group**.

   Add the **HTTP (80) Inbound rules** as shown below, and their source will be **Anywhere**.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

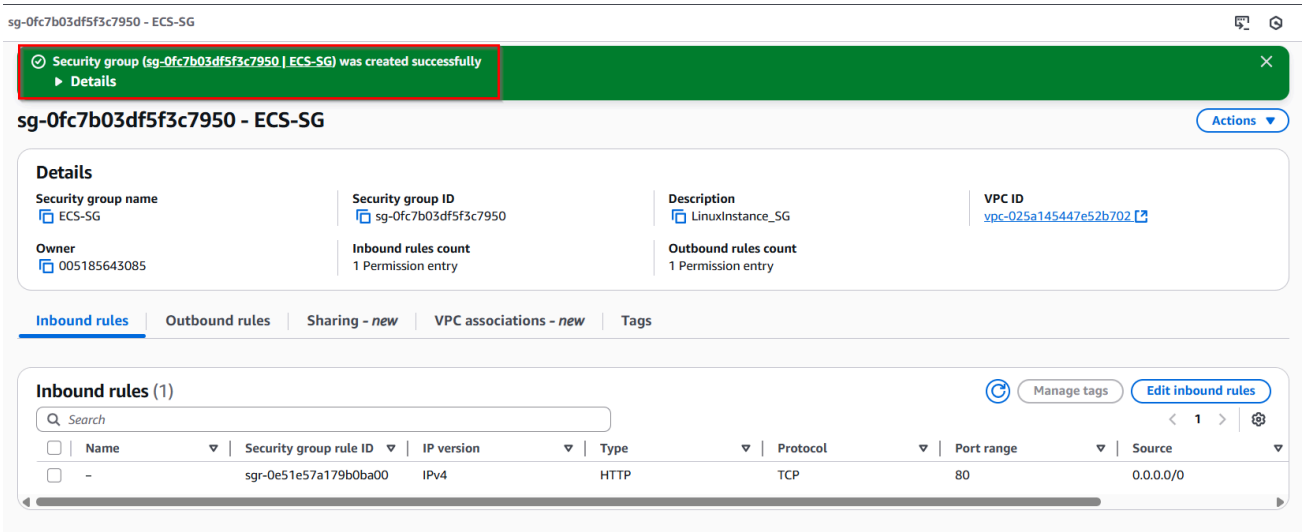


5. Click on **Create Security Group**



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

6. The Security Group will be create.



Thus we have successfully created the Security Group

K21Academy

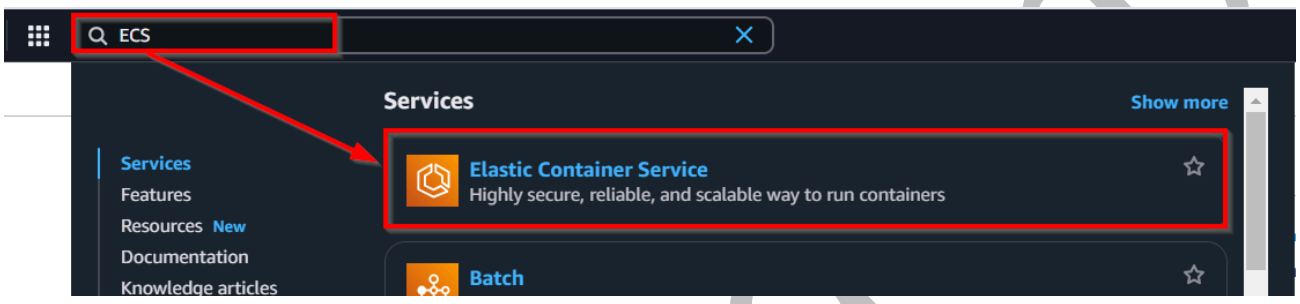
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

7 CREATING ECS CLUSTER

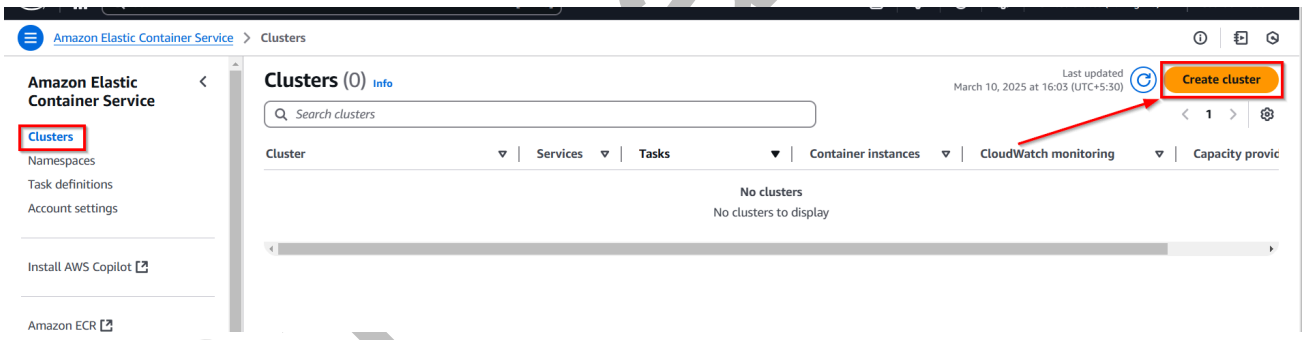
An Amazon ECS cluster is **a logical grouping of tasks or services**. If you are running tasks or services that use the EC2 launch type, a cluster is also a grouping of container instances.

If you are using capacity providers, a cluster is also a logical grouping of capacity providers.

1. In the search box, type **ECS** and click to open it.

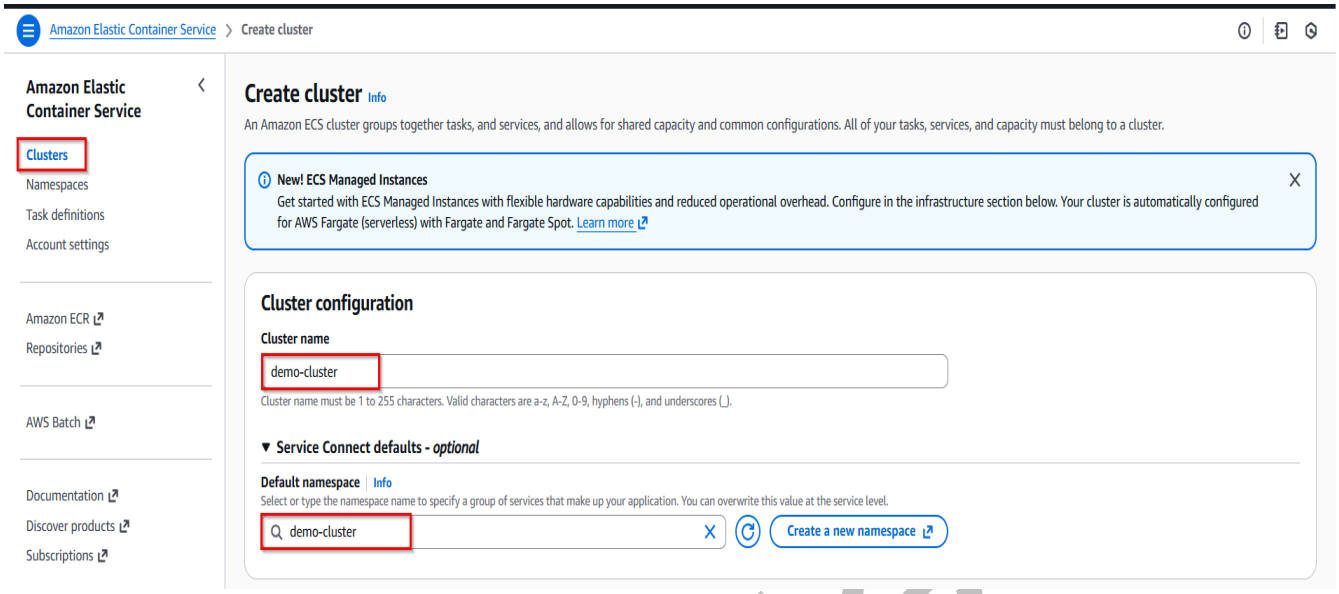


2. On the ECS console, click on **Create Clusters**.



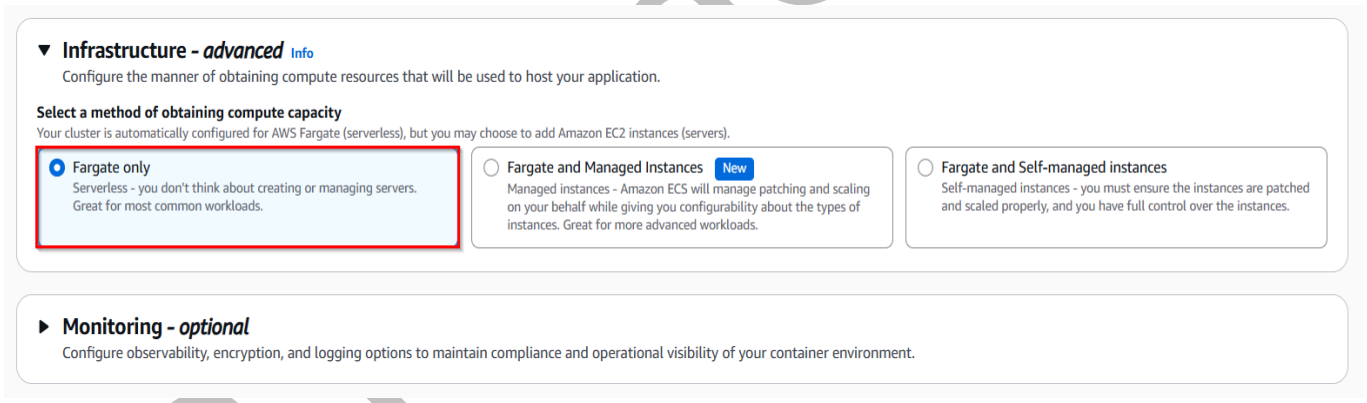
3. Under **Cluster Configuration,** enter the **Cluster name (demo-cluster).**

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

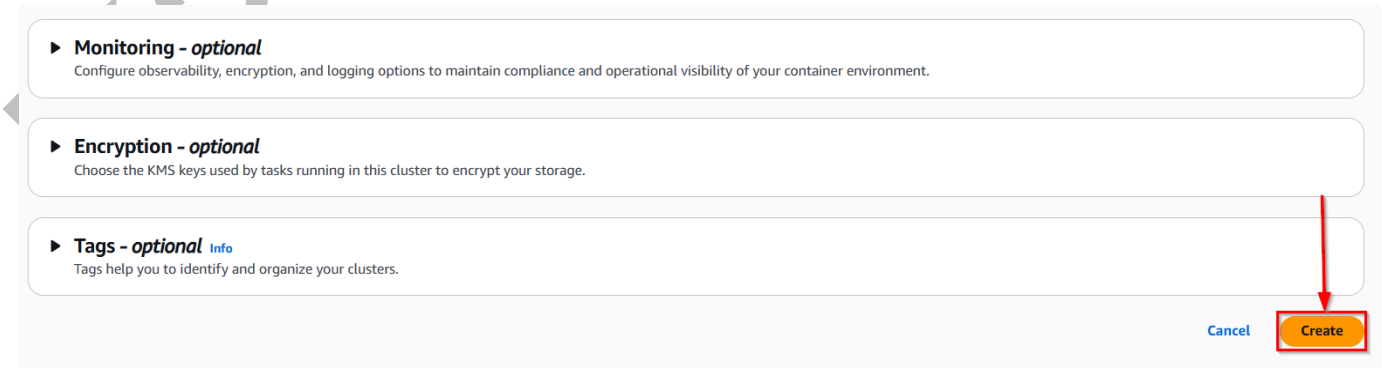


Note: You can give any name to the cluster according to your choice

4. Under Infrastructure, select the AWS Fargate (Serverless).

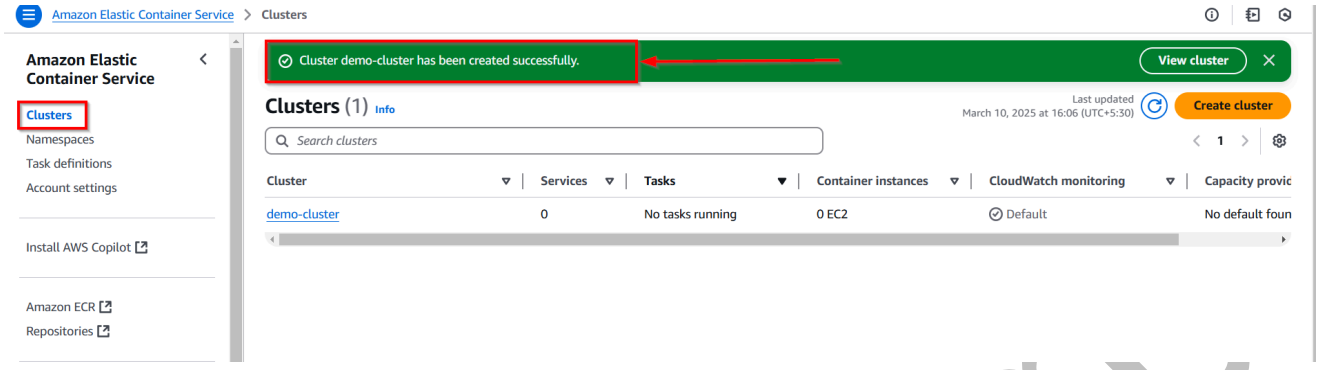


5. Keep everything default and click on **Create**.

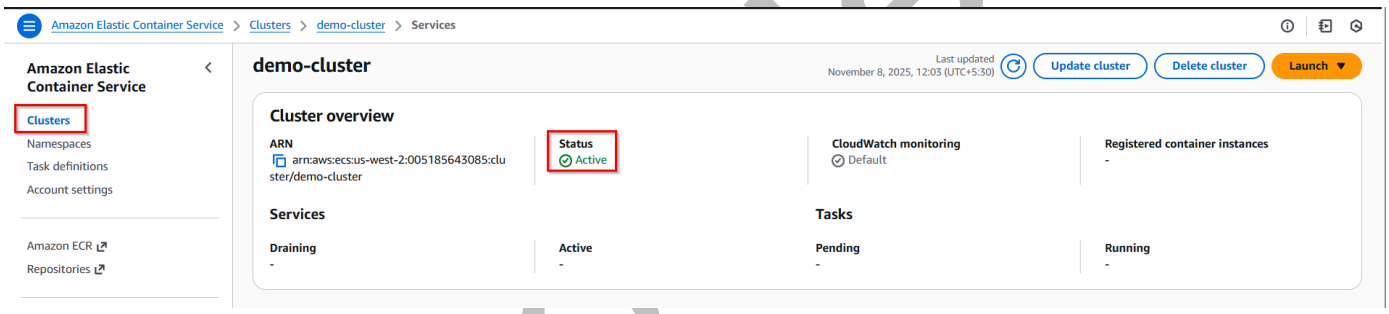


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

6. After creation, your screen will look like this.



7. ECS Instance will take 5-10 minutes to be in Active Status.



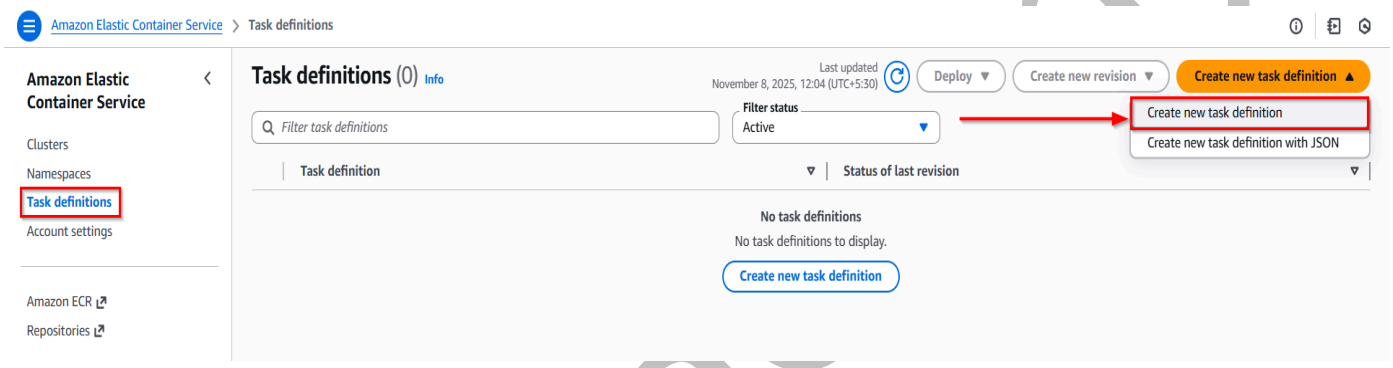
Thus, we have successfully created the ECS Cluster

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

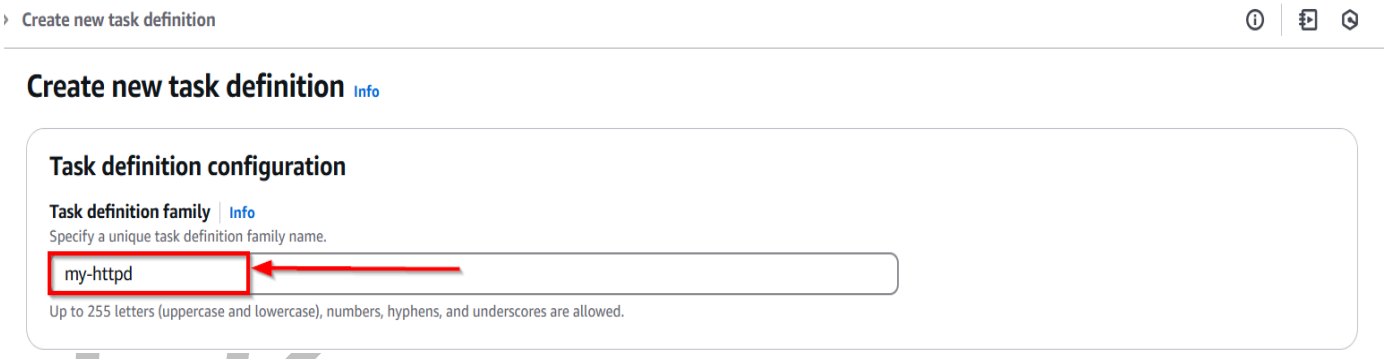
8 CREATING TASK DEFINITIONS

To prepare your application to run on **Amazon ECS**, you must create a task definition. The **task definition** is a text file (in JSON format) that describes one or more containers (up to a maximum of ten) that form your application. The task definition can be thought of as a blueprint for your application.

1. On the ECS **console**, click on **Task Definitions** and then click on **Create new task definition**.

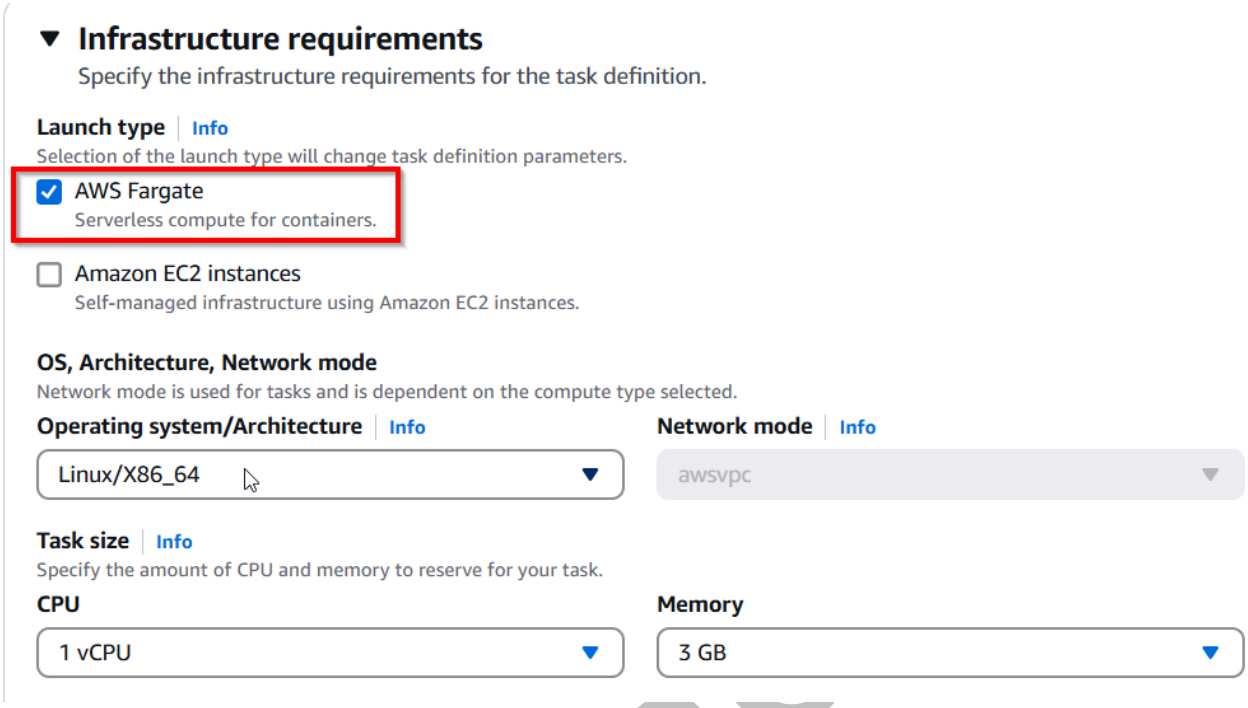


2. Enter the **Task Definition Name** as **my-httpd.**

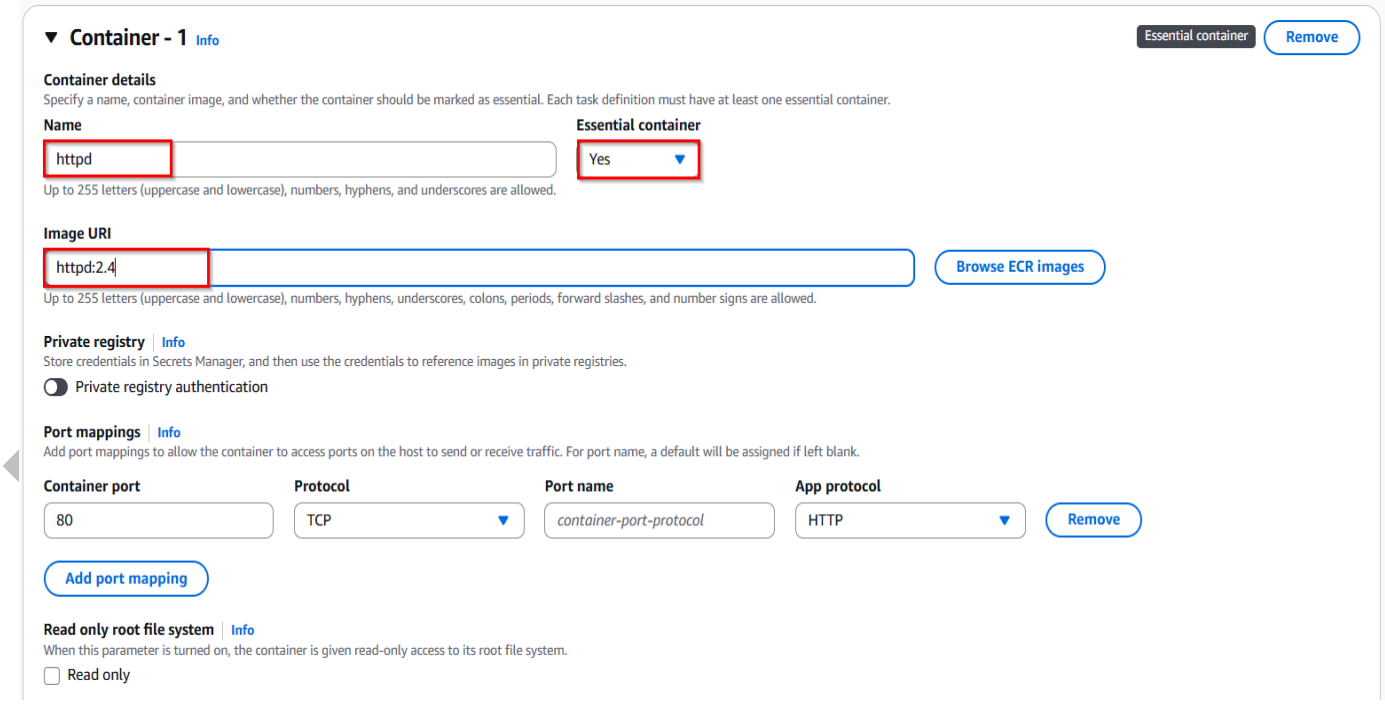


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

3. Under Infrastructure requirements, select the **AWS Fargate.**

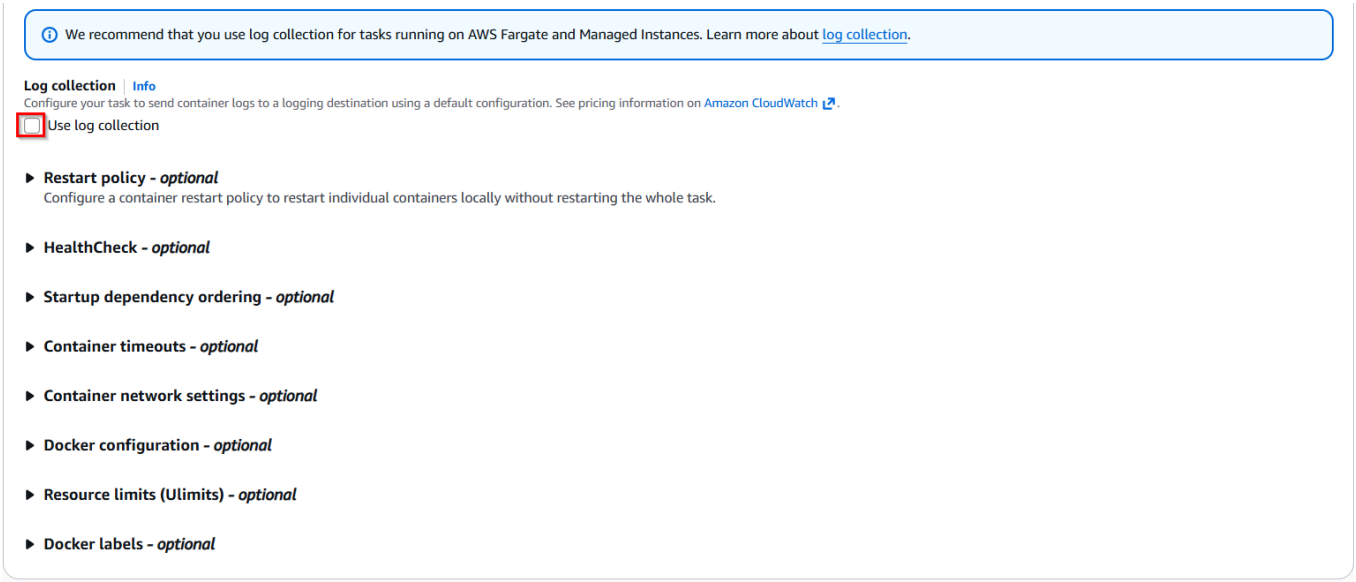


4. Under **Container - 1**, enter the **Container name** as **httpd**, **image name** as **httpd:2.4**

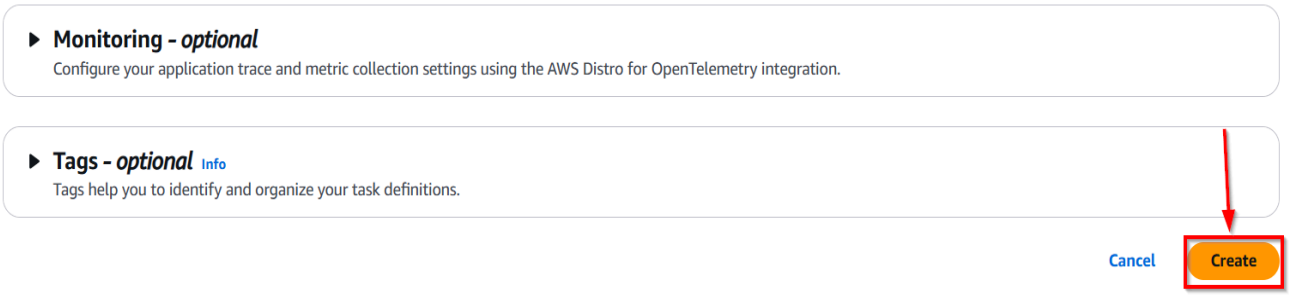


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

5. Scroll down and uncheck the Use Log Collection Option.

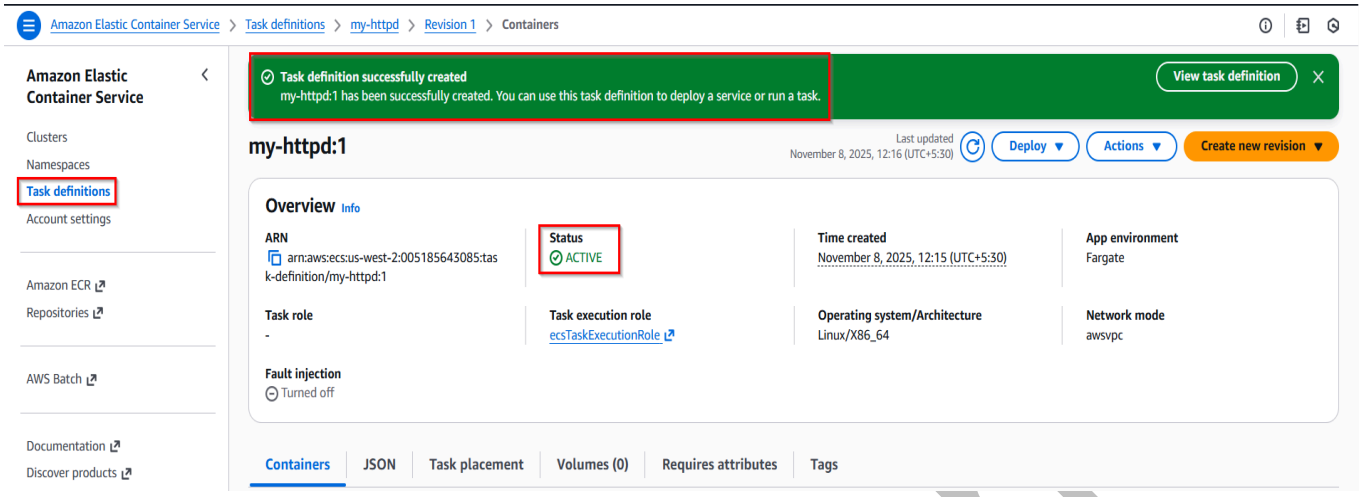


6. Rest keeps everything default, scroll down and click on **Create.**

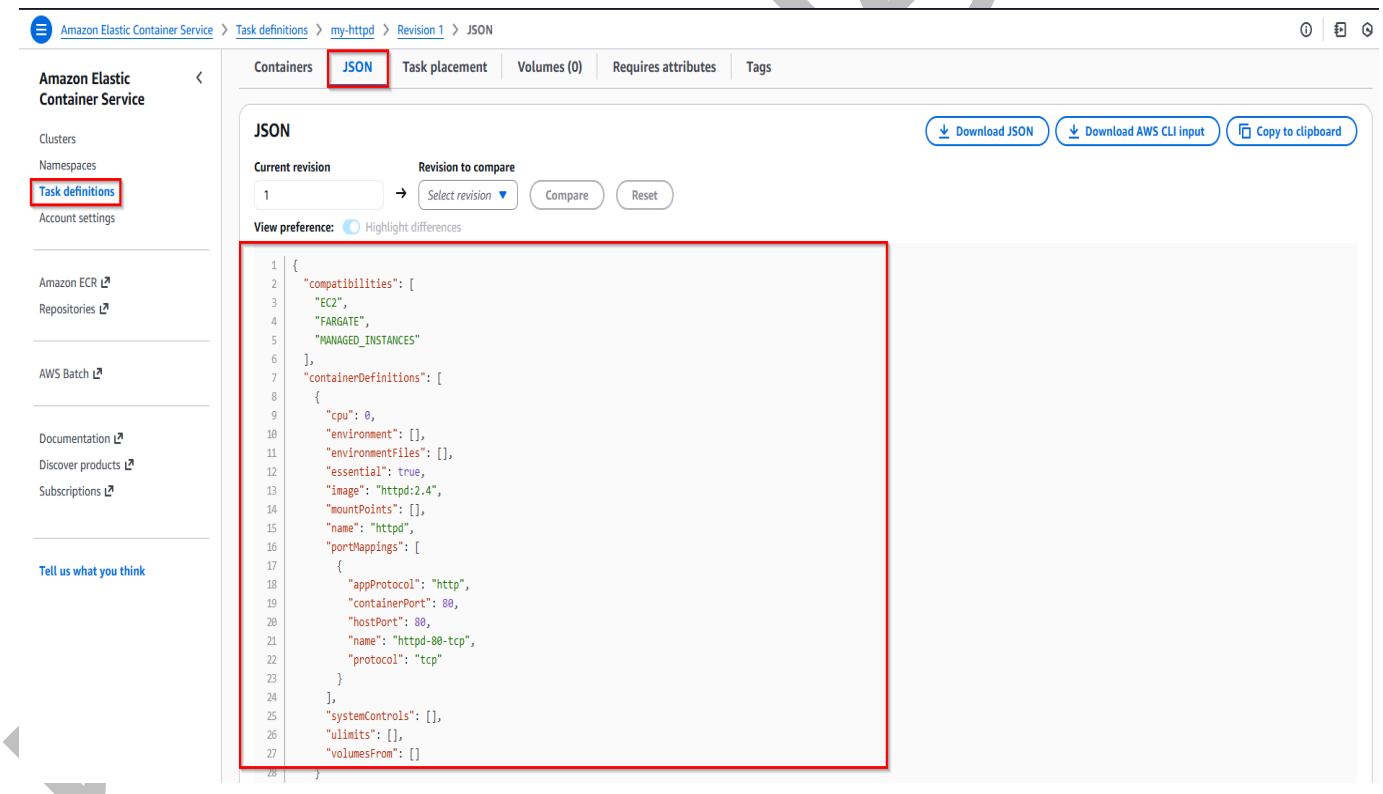


7. **Task definition** will be created successfully.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

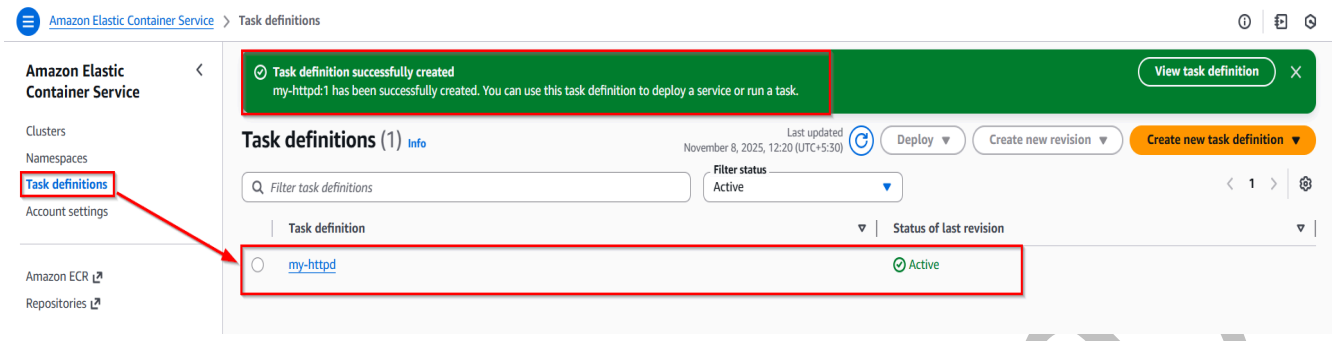


8. Click on **JSON** and you shall see the JSON form of our **Task Definition**.



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

9. Click on **Task Definitions,** and you shall see the one we created.



Thus, we have successfully created the Task Definitions

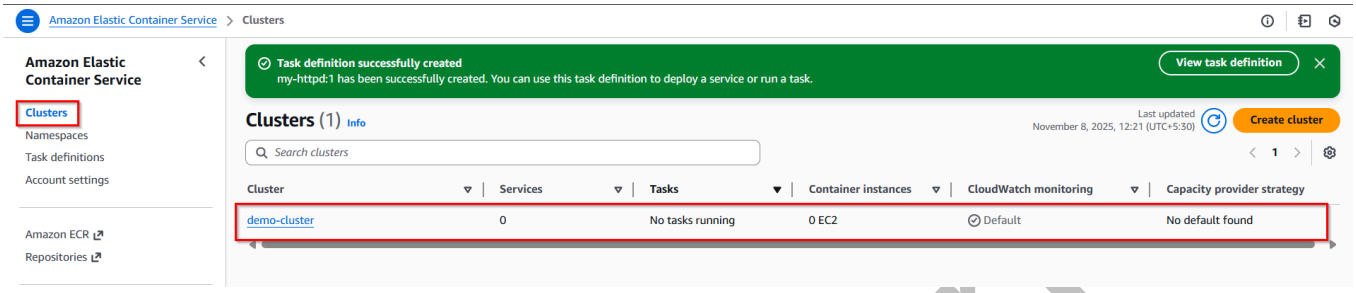
K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

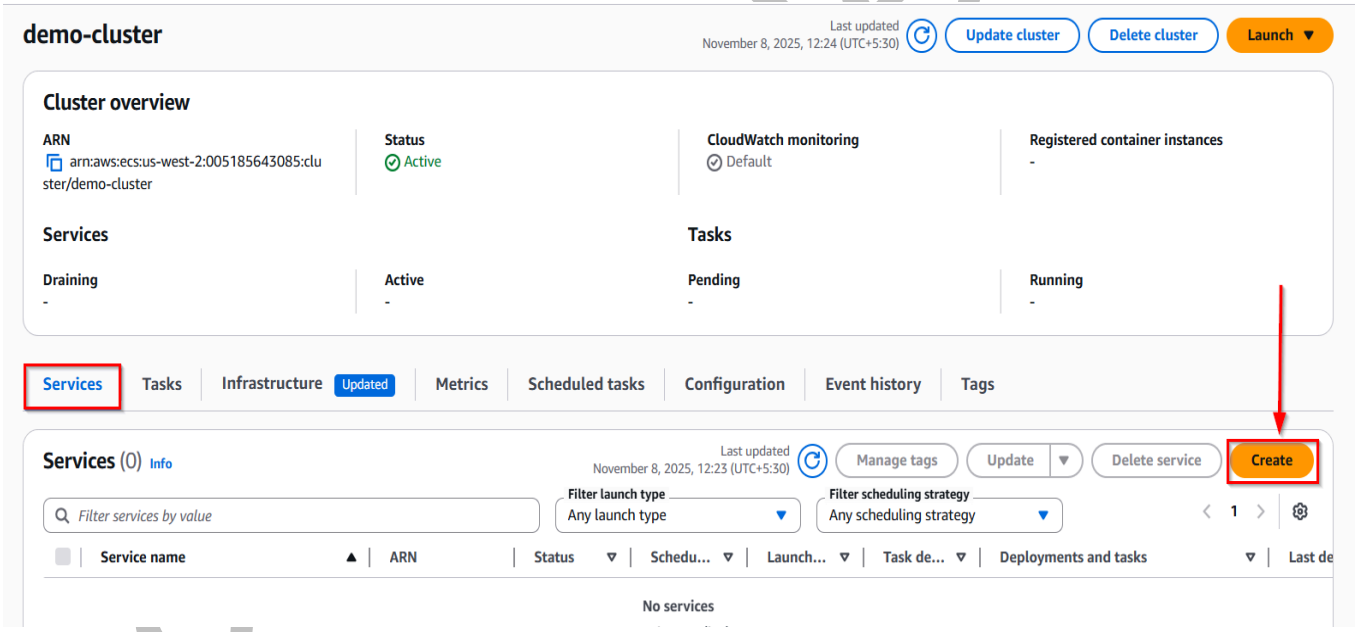
9 CREATE A SERVICE

In this section, we will provide detailed steps to create a service.

1. Switch back on to the **ECS Clusters console** and click on the **cluster** we created earlier.

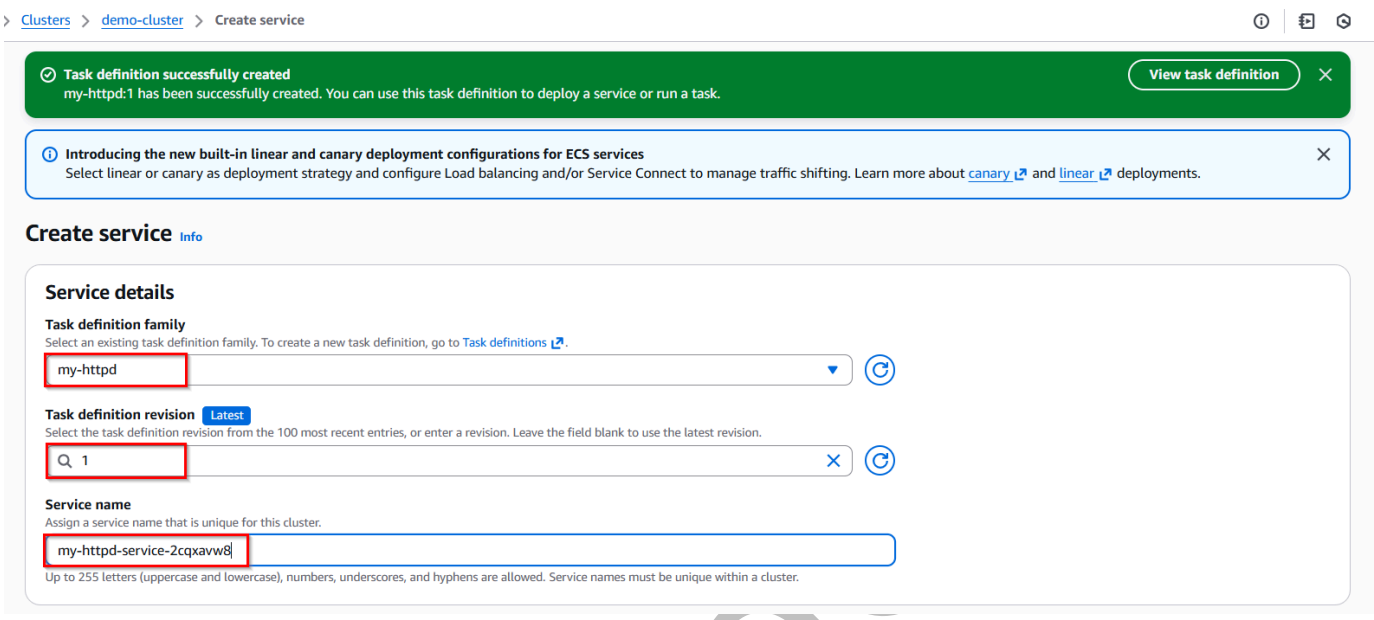


2. Now, under **Services,** click on **Create**.

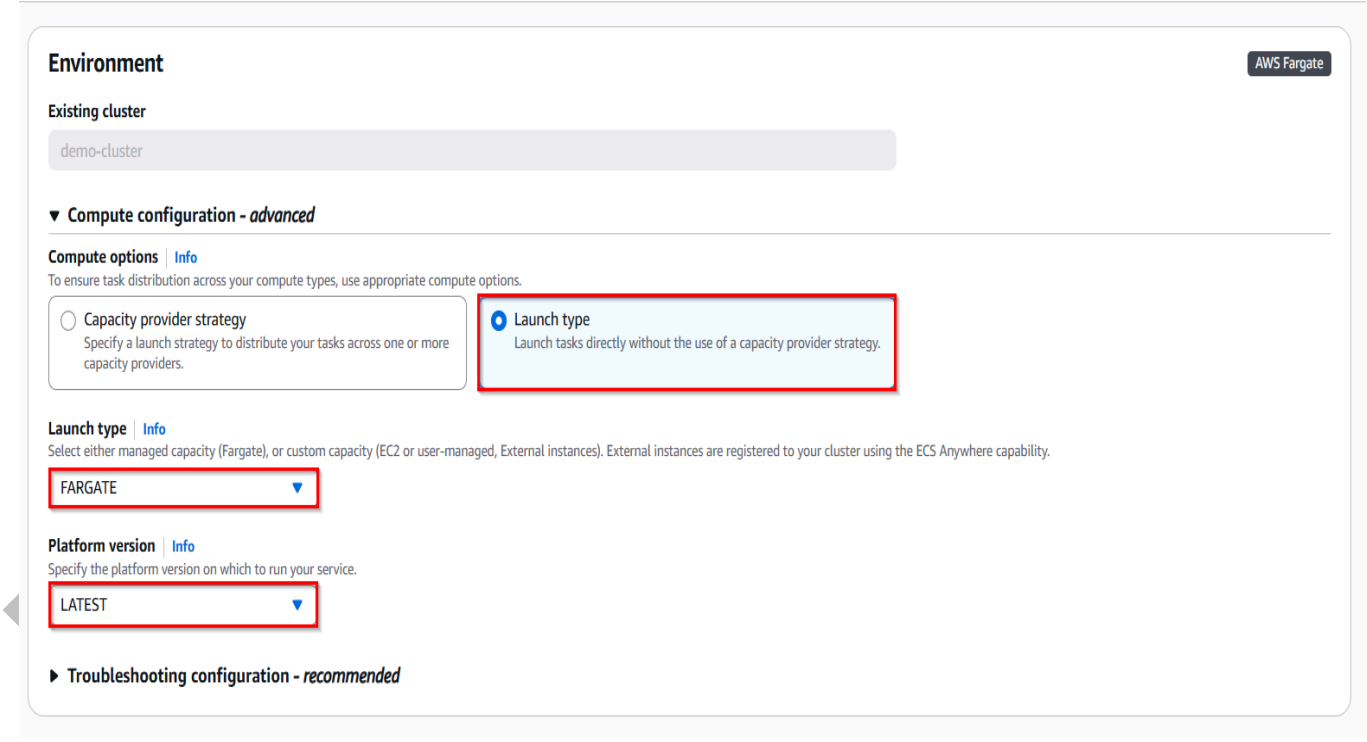


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

3. Under **Service details**, select **my-httpd**. **Under Task definition**, **choose revision 1**.



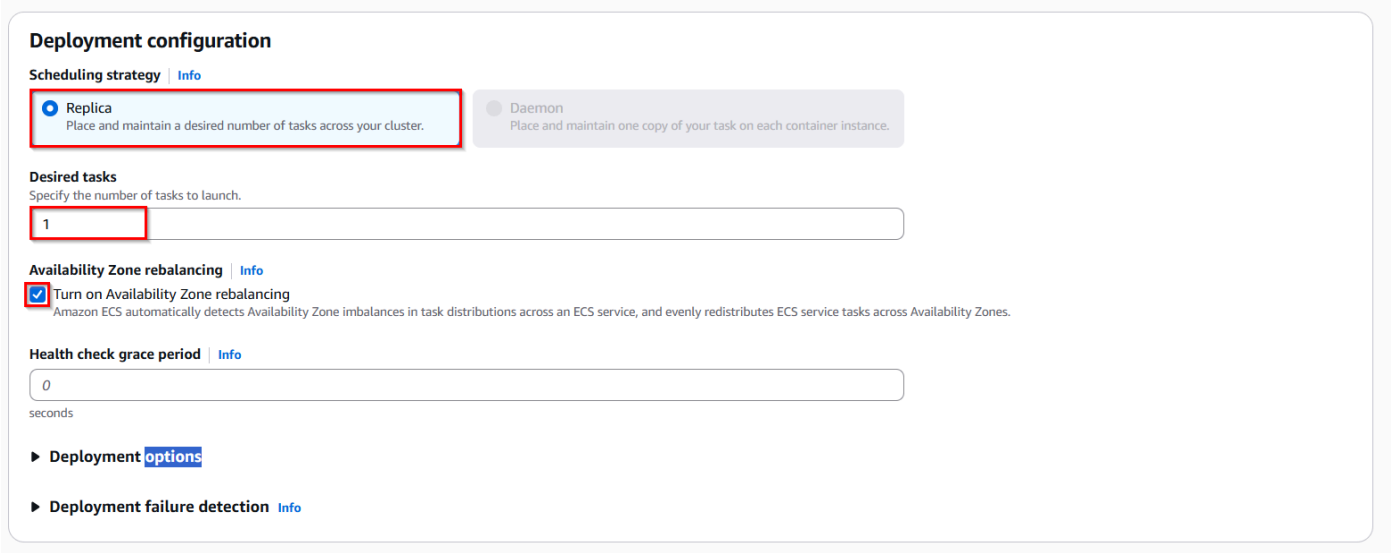
4. Under **Configure Services,** select **Fargate** as the Launch Type.



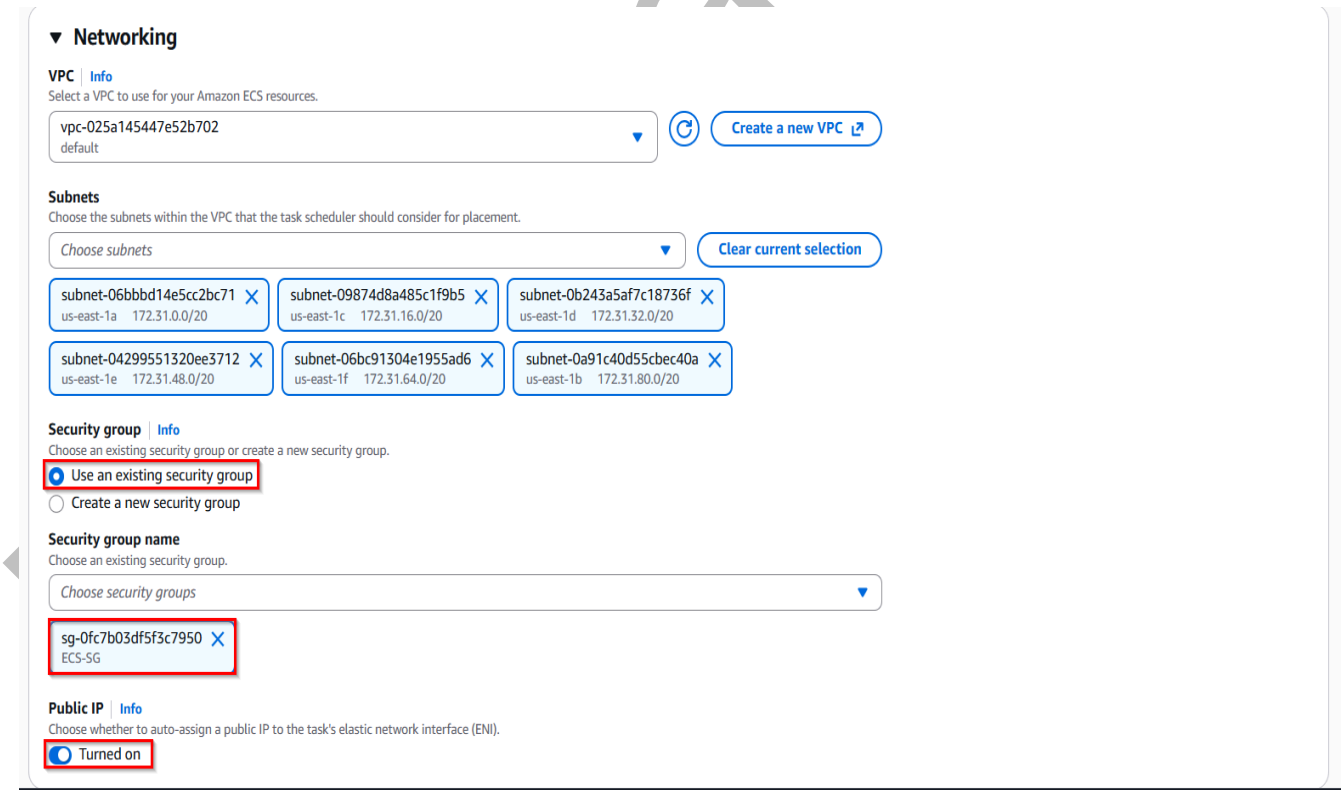
5. Under Deployment Configuration, select the **Task**.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

6. Under **Deployment configuration**, select **Replica** and set the desired number of tasks to **1**.



7. Under Networking, select the (ECS-SG) Security Group (We have created this in the earlier steps).

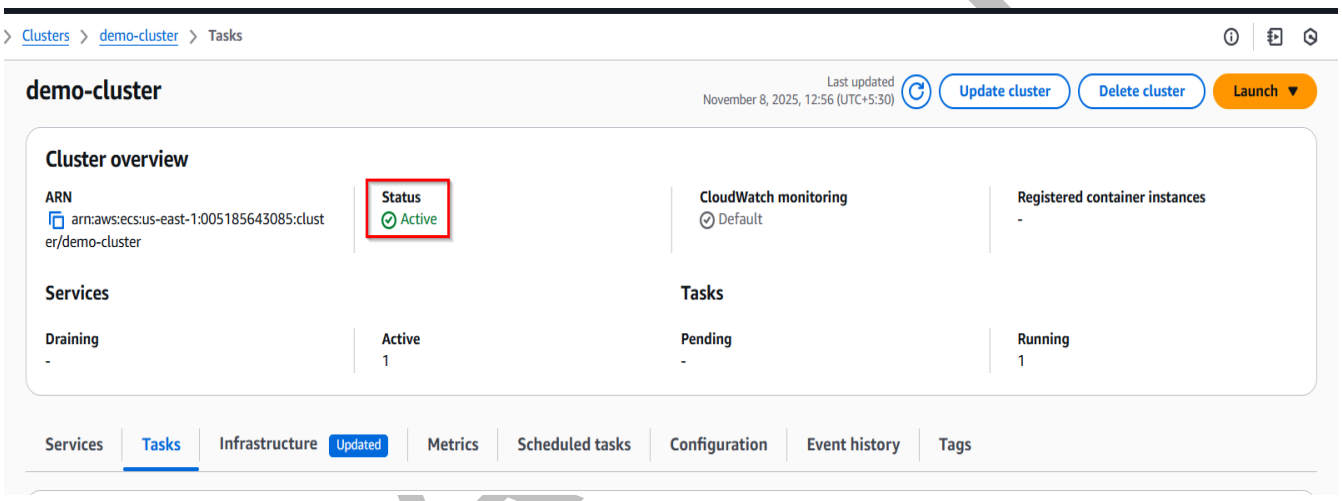


Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

8. Click on **Create.**

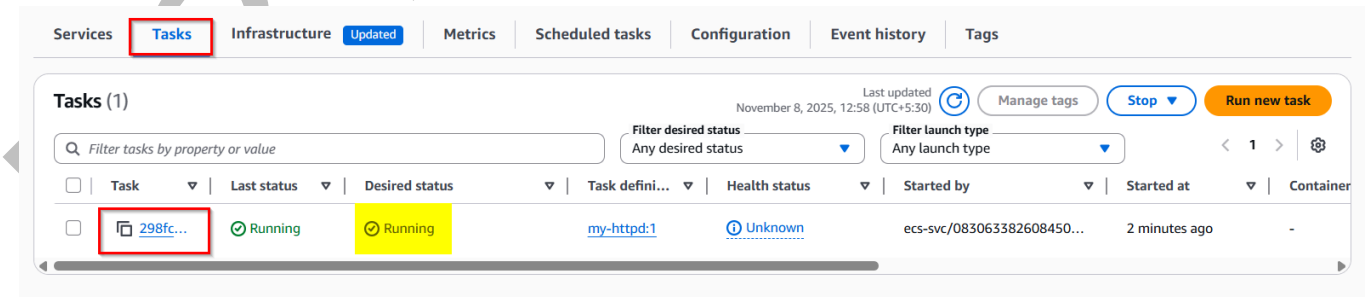


9. You will see a **Task** is launched and is in the **running** state.



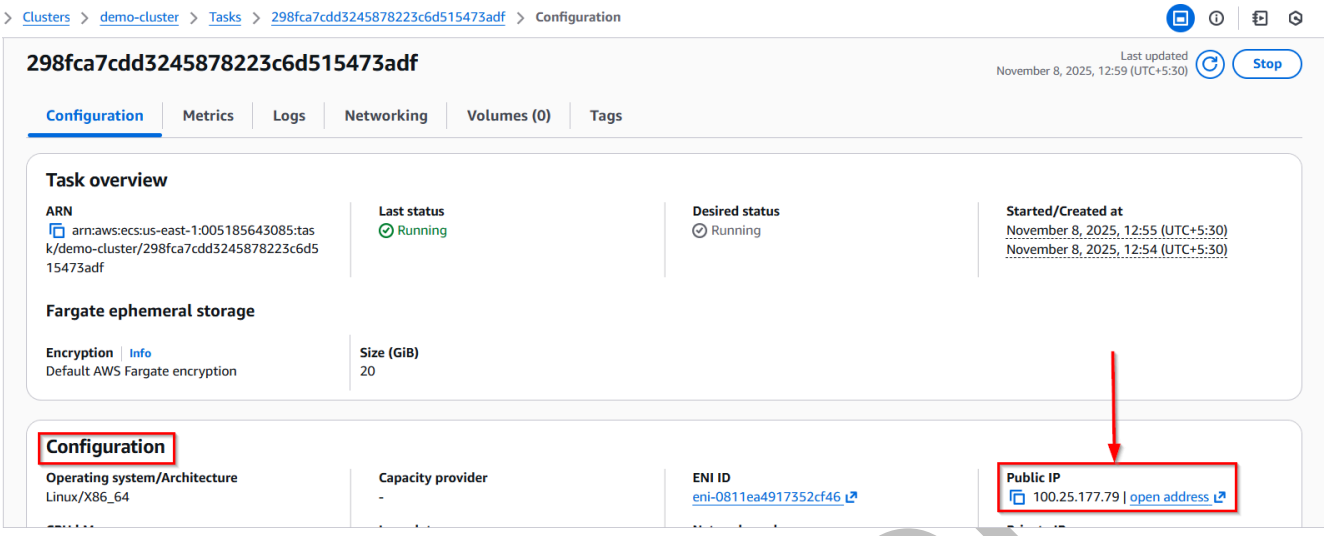
Note: It will take 2-3 minutes to Run the Status

10. Click on **Task** (Make sure it's in the Running State, if not then click on refresh).



11. Here, copy the **public IP** mentioned under Configuration.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.



12. Now paste the **public IP address** in the browser. It will print **It works!**.



It works!

Note: If it does not work, please check your Security group rules that you have added the HTTP port or not.

Thus, we have successfully created the service

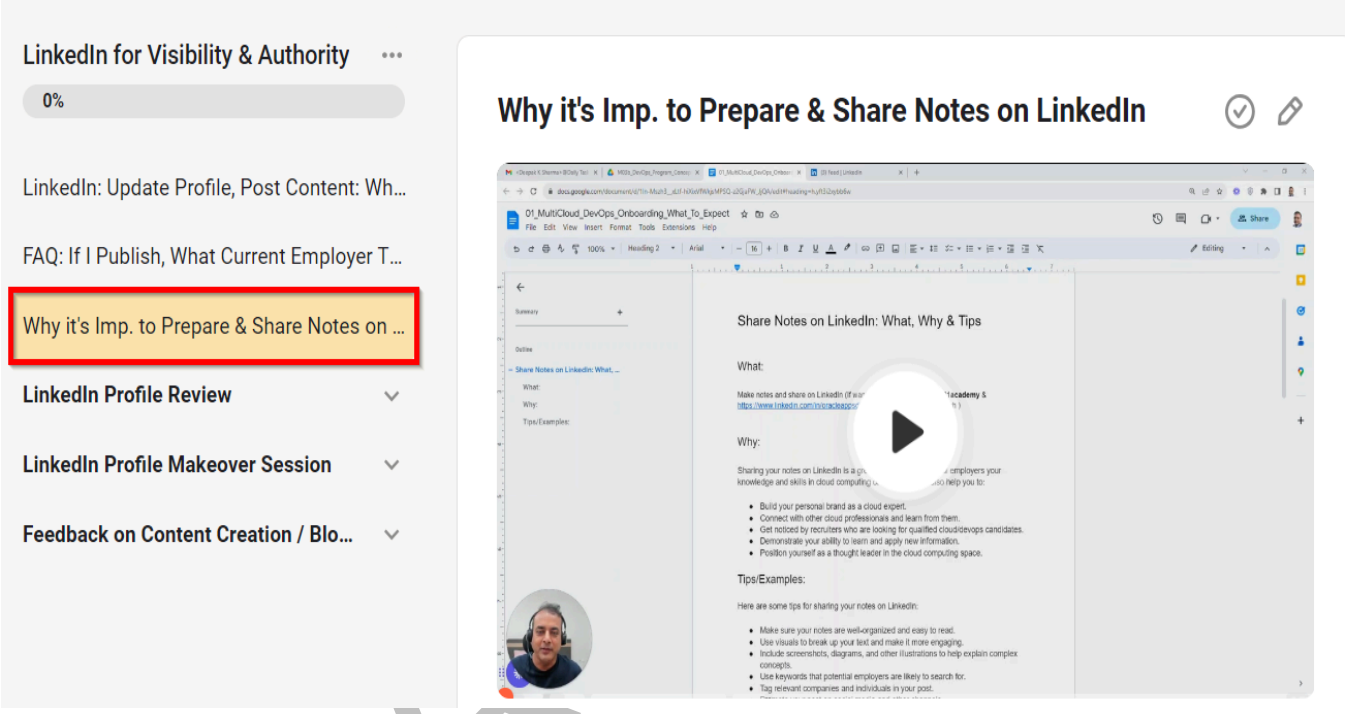
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

10 SHARE YOUR LEARNINGS

In this section, you will learn how to share whatever you learned during this lab on LinkedIn and WhatsApp.

Please watch the below👇 video to understand why it's important to post & how to post:

https://www.skool.com/k21academy/classroom/0c24b6af



👩💻 As you know, building a strong professional profile is essential in today's job market. One way to showcase your skills and knowledge is by sharing your labs and projects on LinkedIn.

Your LinkedIn profile is a powerful tool in your job search. Sharing your labs and projects is just one way to demonstrate your expertise and stand out to potential employers. If you don't have LinkedIn, we strongly recommend you create one for yourself.

📝🖥️ By doing so, you can demonstrate to potential employers or connections that you have hands-on experience in your field and are actively engaged in learning and growing your skills.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

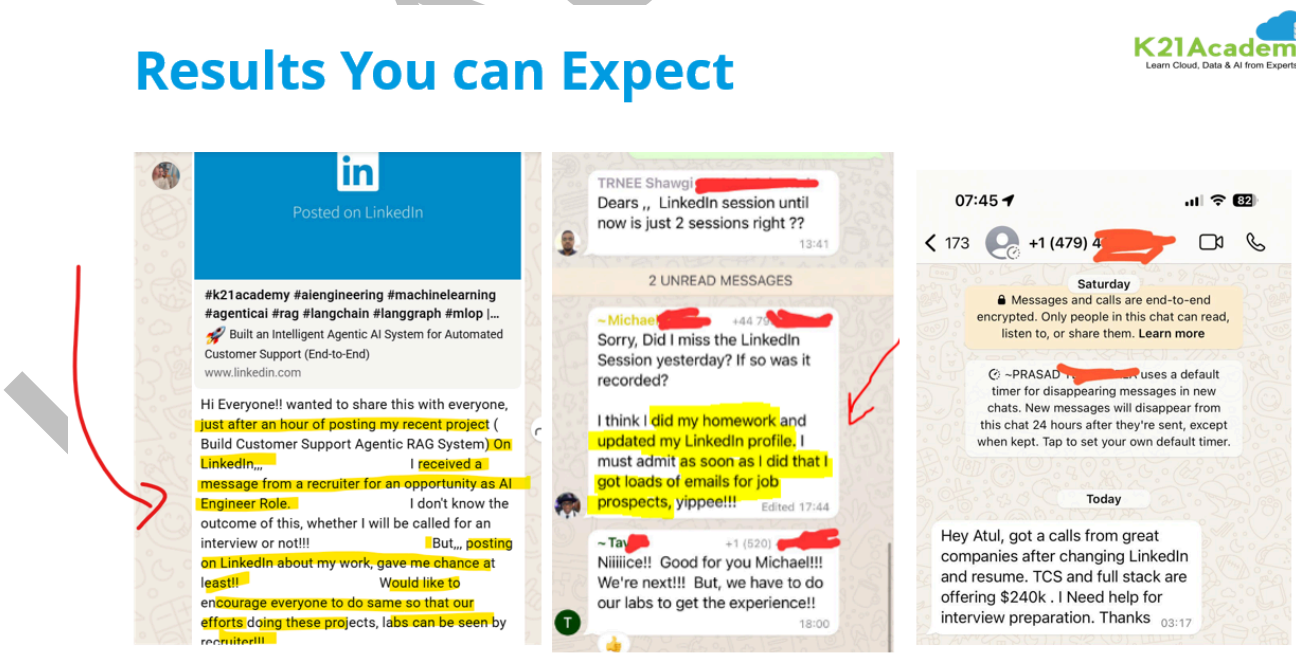
10.1 Share Hands-On Lab on LinkedIn to Build Authority & Job Opportunities

Take a screenshot and share it on your LinkedIn. So, this will attract recruiters and employers to your profile and increase your reach. **Do remember to tag**

This will attract recruiters and potential employers to your profile and increase your Authority & LinkedIn reach.

1. **Do remember to tag so we can share too, and we'll ask other members to share too (this will increase your post reach)**
   a. K21Academy (https://www.linkedin.com/company/k21academy) &
   b. Atul Kumar (https://www.linkedin.com/in/atulk21academy/), as we will further circulate in our network, too, to increase your reach.

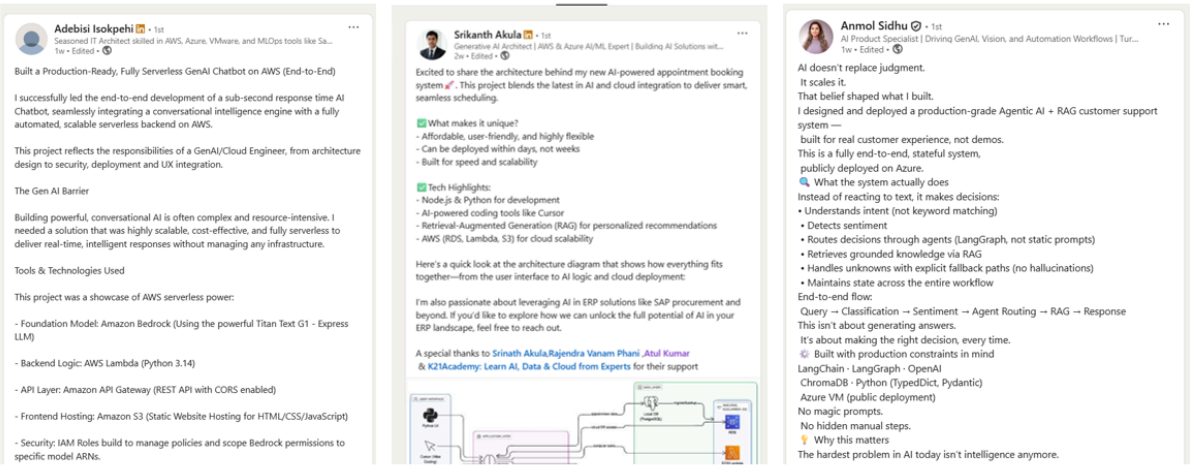
2. Here is what you can expect
   a. Jessy got a message from a recruiter for an AI Engineer Role
   b. Michael got loads of emails for Job prospects
   c. Prasad got calls from great companies that are offering $240K Salary



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

3. Example Links on How to post
    a. Example Posts

Post on LinkedIn for Interview Calls



Adebisi Isokpehi • 1st
Seasoned IT Architect skilled in AWS, Azure, VMware, and MLOps tools like Sa...
1w • Edited •

Built a Production-Ready, Fully Serverless GenAI Chatbot on AWS (End-to-End)

I successfully led the end-to-end development of a sub-second response time AI Chatbot, seamlessly integrating a conversational intelligence engine with a fully automated, scalable serverless backend on AWS.

This project reflects the responsibilities of a GenAI/Cloud Engineer, from architecture design to security, deployment and UX integration.

The Gen AI Barrier

Building powerful, conversational AI is often complex and resource-intensive. I needed a solution that was highly scalable, cost-effective, and fully serverless to deliver real-time, intelligent responses without managing any infrastructure.

Tools & Technologies Used

This project was a showcase of AWS serverless power:

- Foundation Model: Amazon Bedrock (Using the powerful Titan Text G1 - Express LLM)

- Backend Logic: AWS Lambda (Python 3.14)

- API Layer: Amazon API Gateway (REST API with CORS enabled)

- Frontend Hosting: Amazon S3 (Static Website Hosting for HTML/CSS/JavaScript)

- Security: IAM Roles build to manage policies and scope Bedrock permissions to specific model ARNs.

Srikanth Akula • 1st
Generative AI Architect | AWS & Azure AI/ML Expert | Building AI Solutions wit...
2w • Edited •

Excited to share the architecture behind my new AI-powered appointment booking system 🚀. This project blends the latest in AI and cloud integration to deliver smart, seamless scheduling.

✅ What makes it unique?
- Affordable, user-friendly, and highly flexible
- Can be deployed within days, not weeks
- Built for speed and scalability

✅ Tech Highlights:
- Node.js & Python for development
- AI-powered coding tools like Cursor
- Retrieval-Augmented Generation (RAG) for personalized recommendations
- AWS (RDS, Lambda, S3) for cloud scalability

Here's a quick look at the architecture diagram that shows how everything fits together—from the user interface to AI logic and cloud deployment:

I'm also passionate about leveraging AI in ERP solutions like SAP procurement and beyond. If you'd like to explore how we can unlock the full potential of AI in your ERP landscape, feel free to reach out.

A special thanks to Srinath Akula,Rajendra Vanam Phani ,Atul Kumar & K21Academy: Learn AI, Data & Cloud from Experts for their support

Anmol Sidhu • 1st
AI Product Specialist | Driving GenAI, Vision, and Automation Workflows | Tur...
1w • Edited •

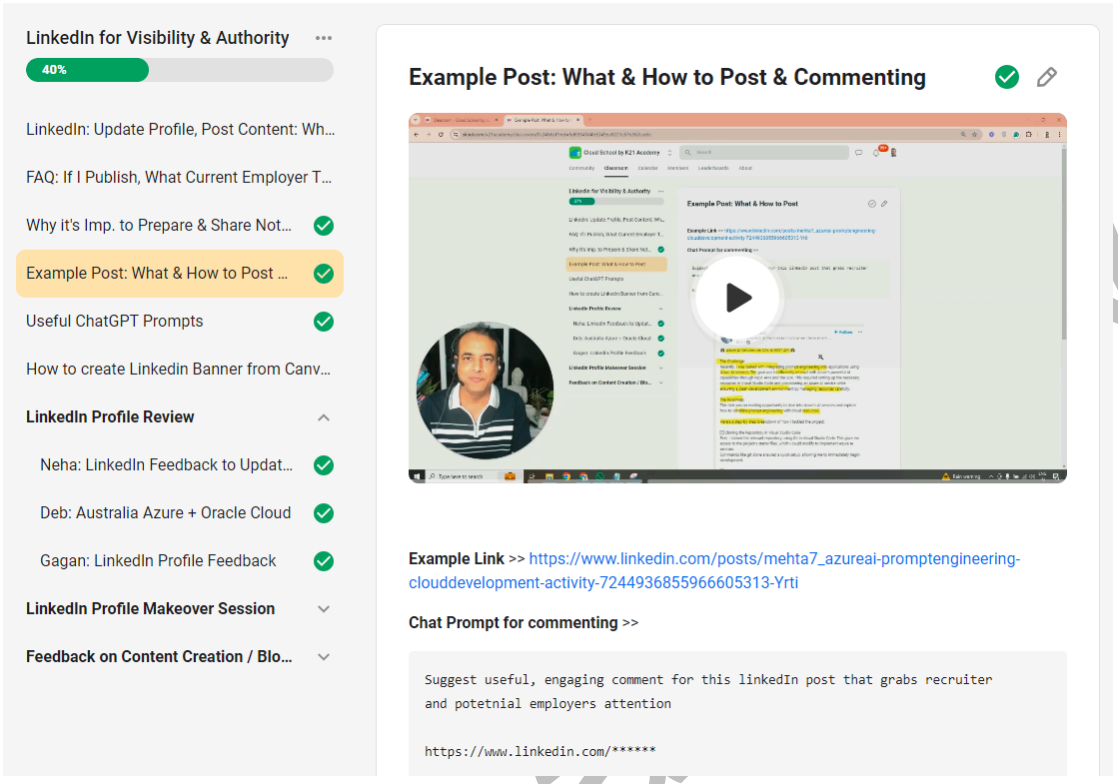
AI doesn't replace judgment.
It scales it.
That belief shaped what I built.
I designed and deployed a production-grade Agentic AI + RAG customer support system —
built for real customer experience, not demos.
This is a fully end-to-end, stateful system,
publicly deployed on Azure.
🔍 What the system actually does
Instead of reacting to text, it makes decisions:
• Understands intent (not keyword matching)
• Detects sentiment
• Routes decisions through agents (LangGraph, not static prompts)
• Retrieves grounded knowledge via RAG
• Handles unknowns with explicit fallback paths (no hallucinations)
• Maintains state across the entire workflow
End-to-end flow:
Query → Classification → Sentiment → Agent Routing → RAG → Response
This isn't about generating answers.
It's about making the right decision, every time.
⚙️ Built with production constraints in mind
LangChain · LangGraph · OpenAI
ChromaDB · Python (TypedDict, Pydantic)
Azure VM (public deployment)
No magic prompts.
No hidden manual steps.
💡 Why this matters
The hardest problem in AI today isn't intelligence anymore.

    b. Example Video & Post

K21Ca

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

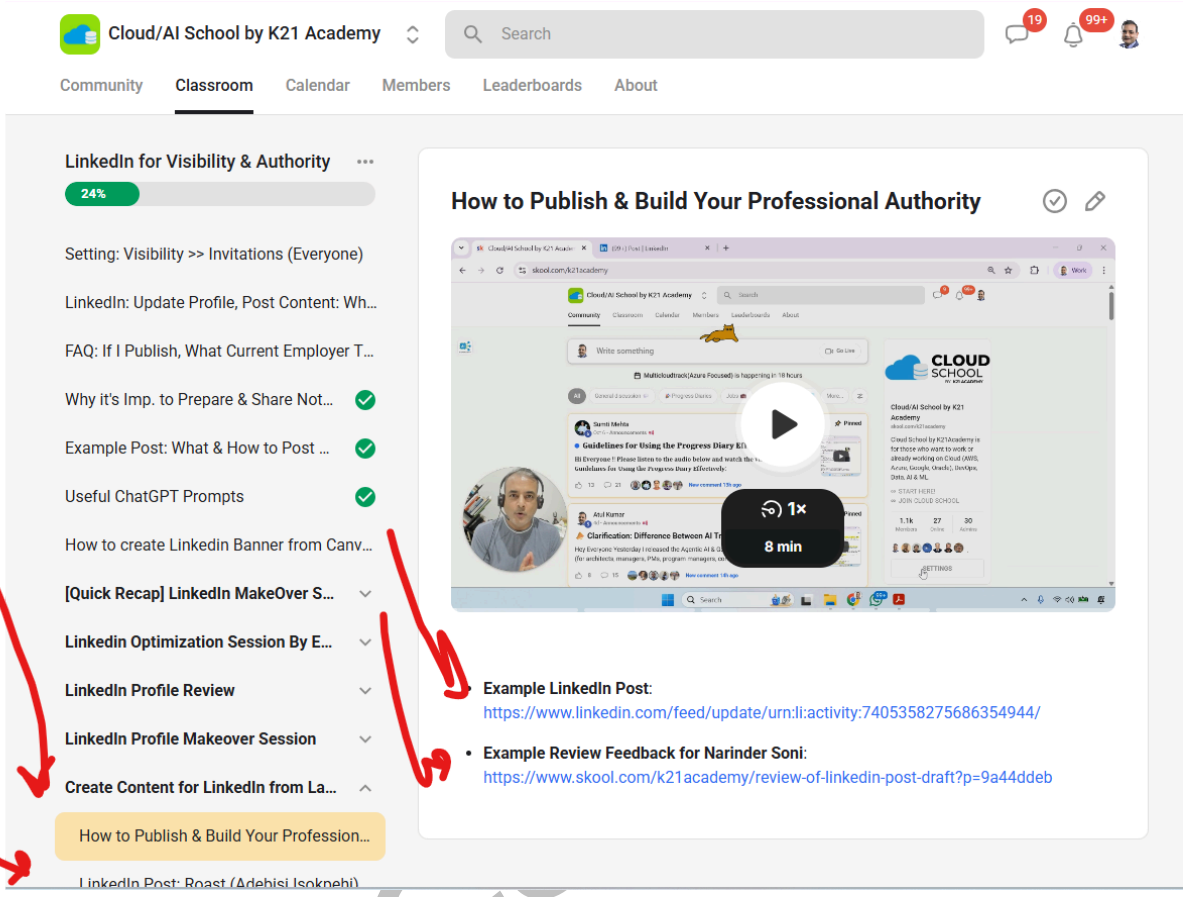
https://www.skool.com/k21academy/classroom/0c24b6af



c. Check few more linkedIn post examples (do's and don'ts): https://www.skool.com/k21academy/classroom/0c24b6af?md=edfe6092fbb84f6bba03ceb439d1e00a

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.



10.2 Share win in the Community

Share screenshots in Cloud School Community under your **'Progress Diary'**

If you don't know How to Access Your Progress Diary on Skool Portal check this:

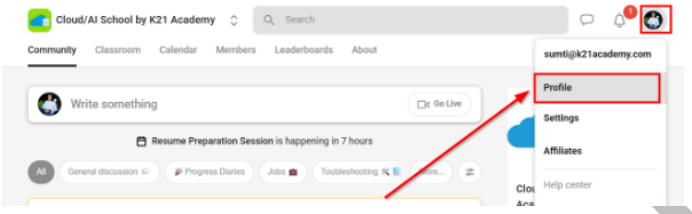
http://k21academy.com/FindYourDiary

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

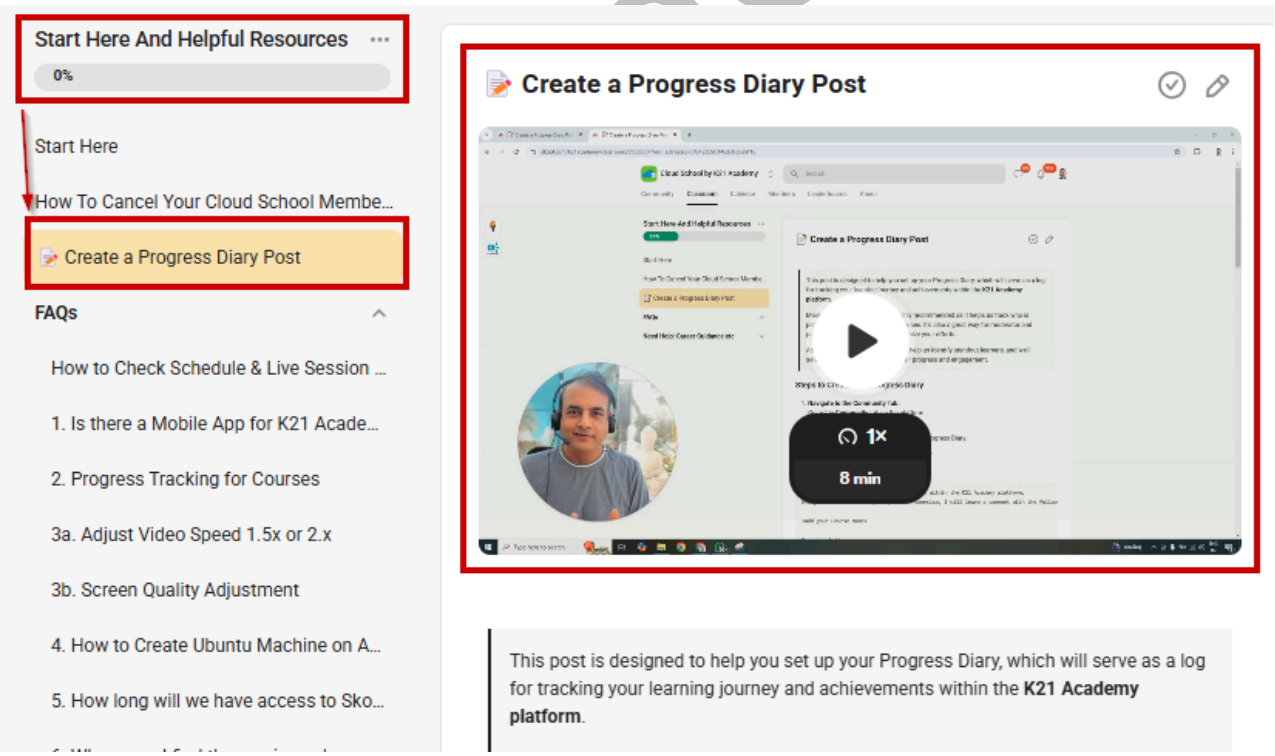
How to Access Your Progress Diary on Skool Portal

How to Access and Manage Your Progress Diary on Skool Portal

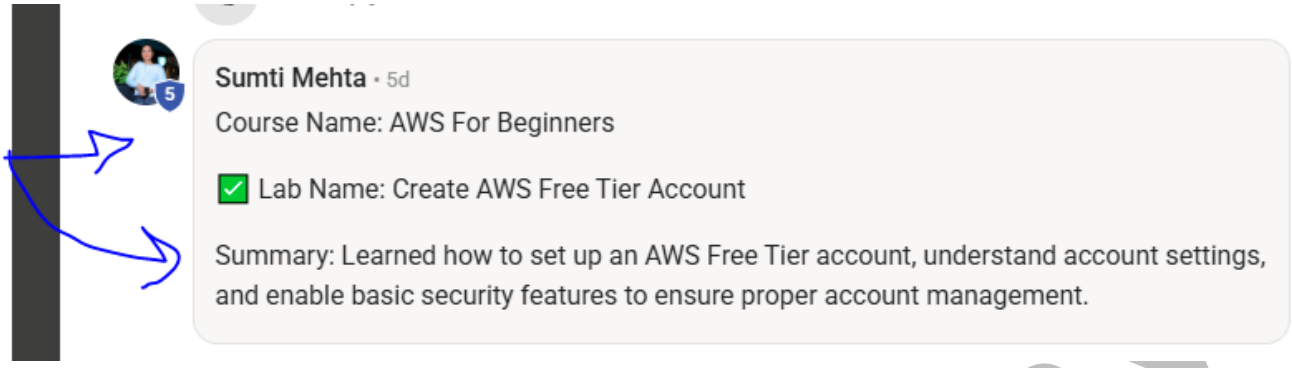
1. **Log In to Skool Portal:**
   - Go to the **Skool** website (or the link provided to you https://www.skool.com/k21academy).
   - Log in with your credentials (email/username and password).
2. **Navigate to Your Profile:**
   - Once logged in, click on your **profile picture** or **name** at the top right corner of the screen.



This will boost your confidence in the progressive path you are following, and encourage/inspire others to perform these hands-on labs.



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.



K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

11 DELETING/CLEANUP

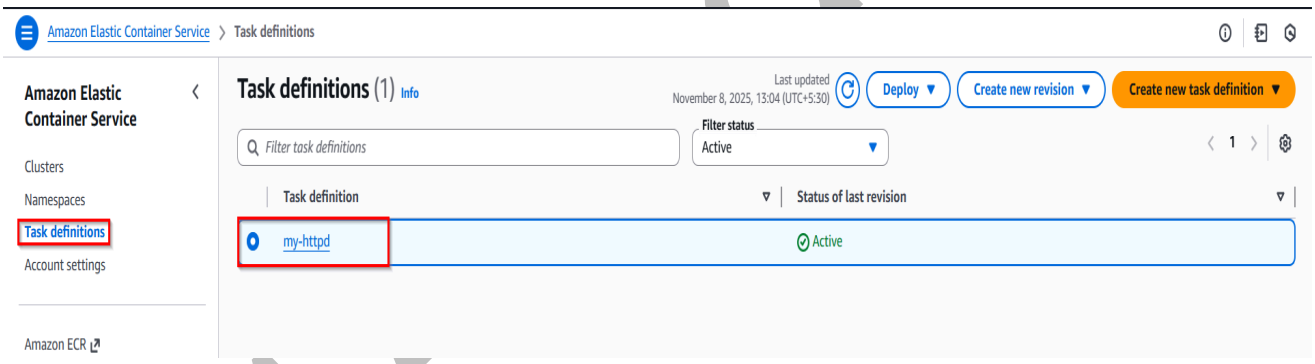
In this section, we will be deleting all the resources we have used for this lab in order to be in the free tier limit, as keeping these resources running will be chargeable.

Note: Here we used the AWS Fargate Service, which is a chargeable service. So, we suggest you please delete all the resources once you have completed the Activity Guide.

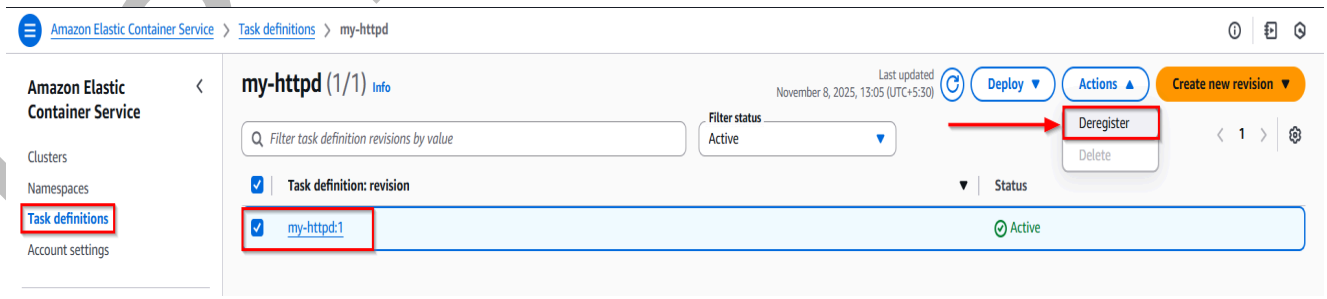
11.1 Deregister the Tasks

In this section, we will provide the detailed steps to deregister the Tasks

1. Now, go back to ECS Console and click on the **Task definition** that you created.

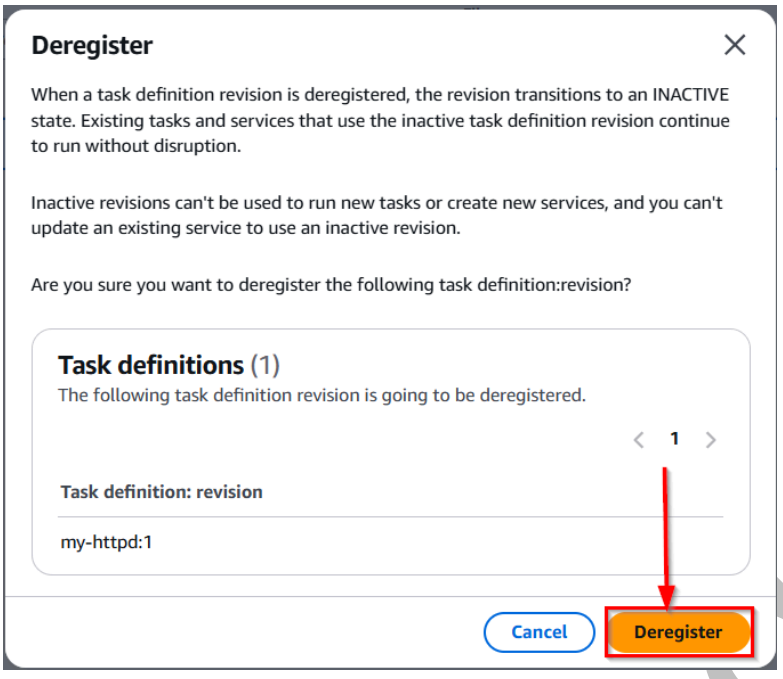


2. Under **Actions**, Click on **Deregister.**



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

3. Click on **Deregister**.



Thus, we have successfully deregistered the Task.

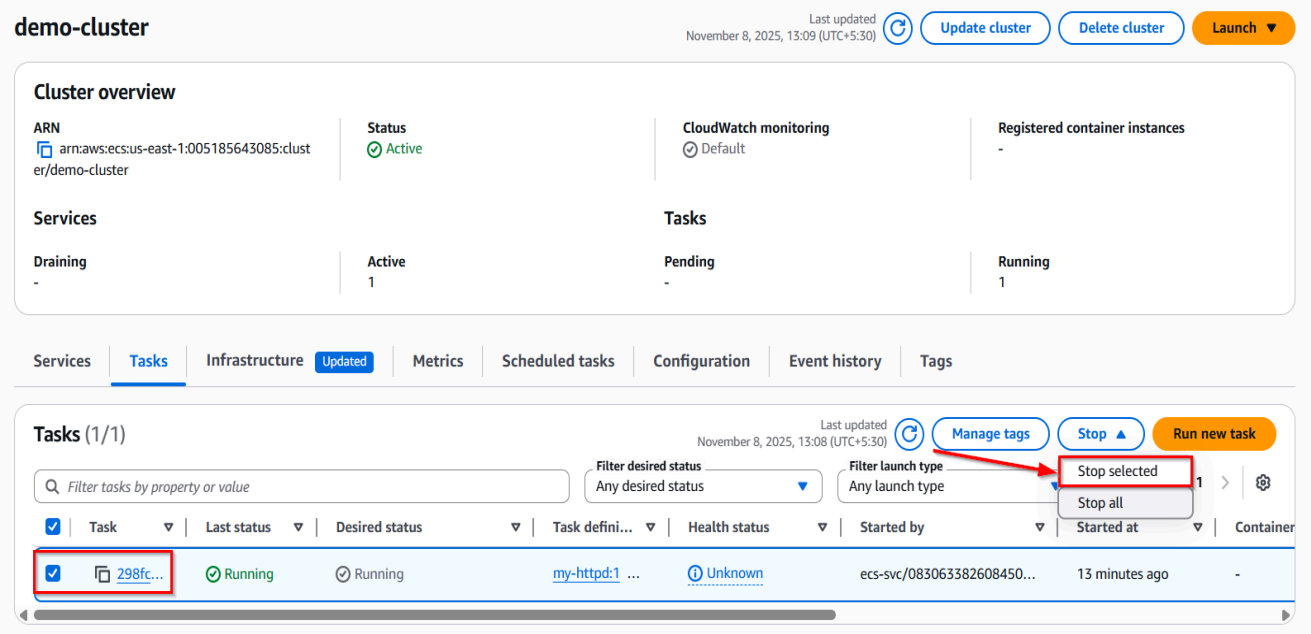
11.2 Deleting the Cluster

In this section, we will be providing detailed steps to delete the Cluster

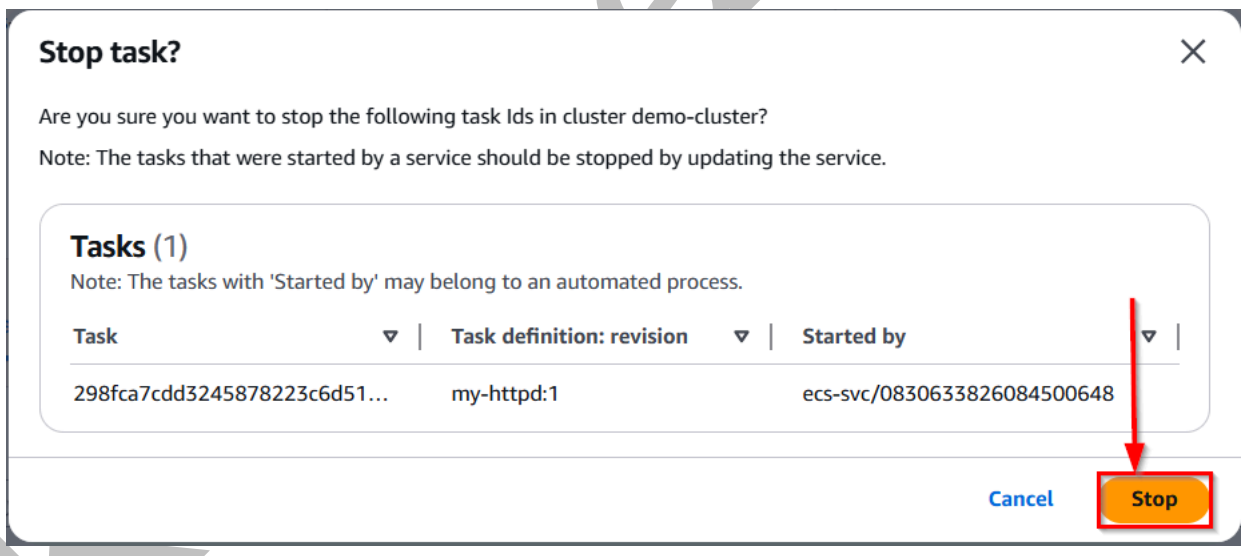
K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

1. Select the **Task** and click on **Stop Selected**.

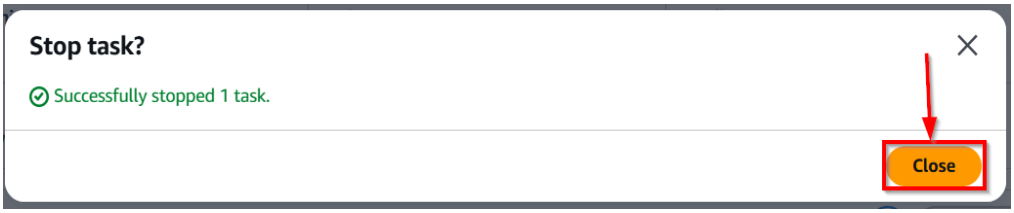


2. Click on **Stop**.

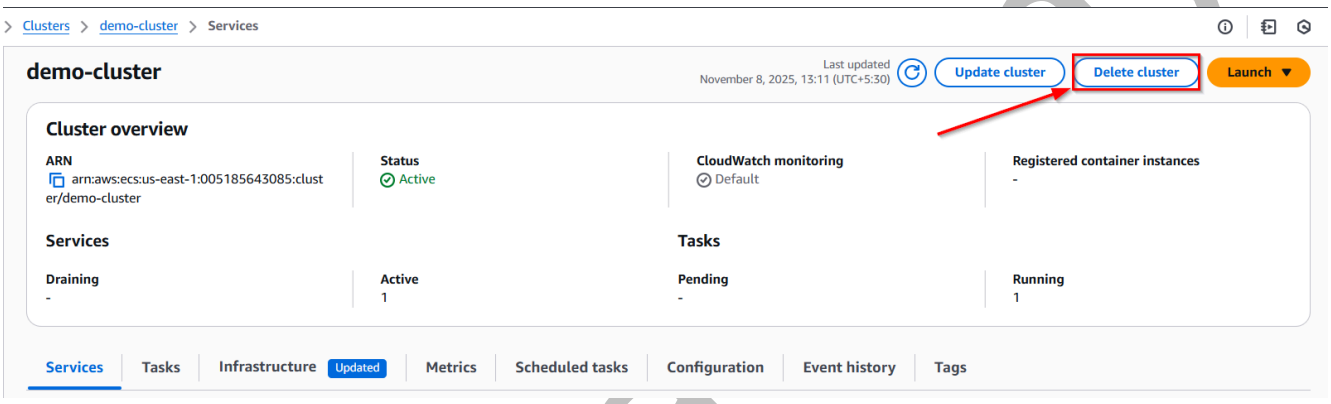


3. Click on **Close**.

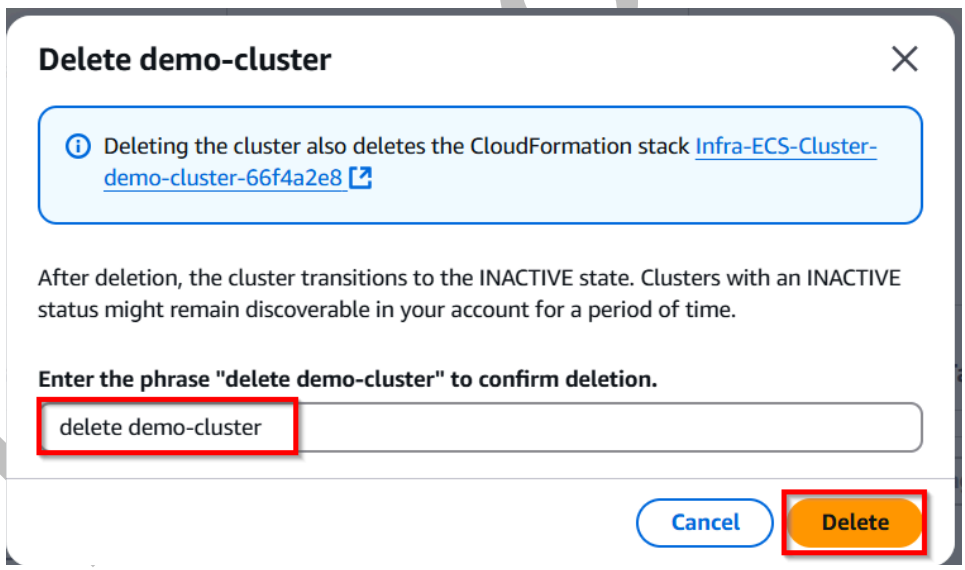
Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.



4. Click on **Delete Cluster.**



5. Type **delete demo-cluster** and click on **Delete**. The Cluster will be deleted in some time



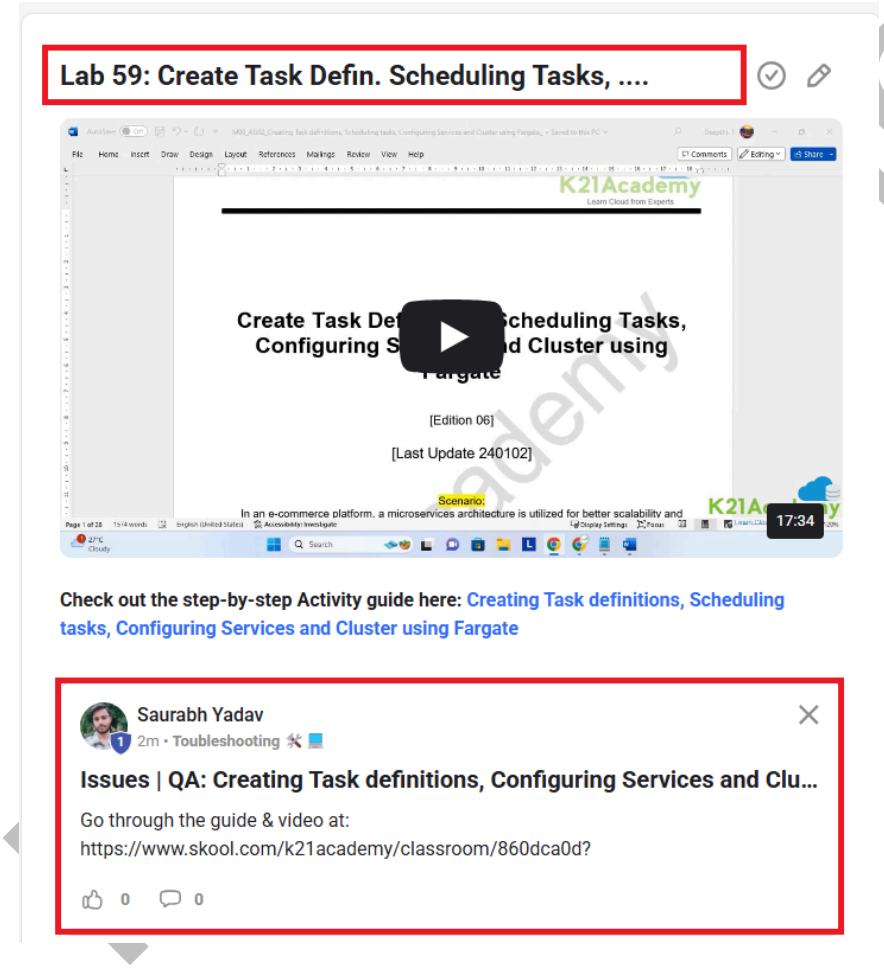
Thus, we have successfully deleted the cluster.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

12 TROUBLESHOOTING

In this section, we outline some common issues you may encounter during the process, along with explanations of why these problems may occur. This will help you quickly identify and resolve any challenges you face.

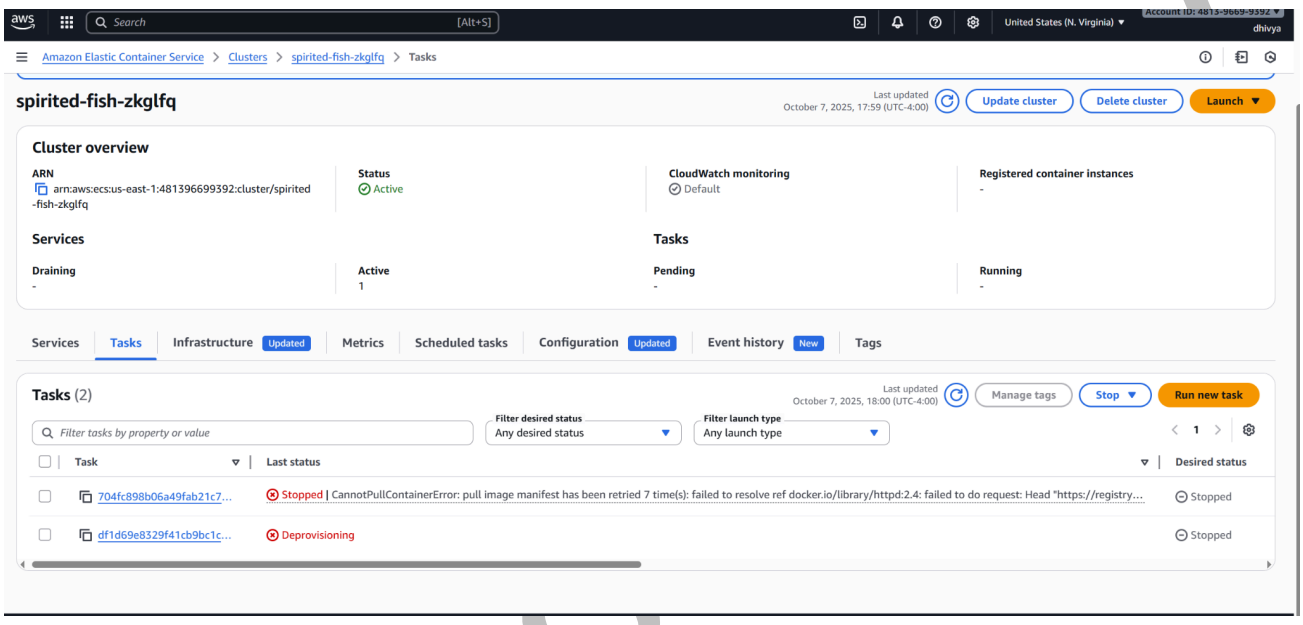
Note: If you hit any issue in performing this lab, then add your issue in the thread below this lab (linked to this thread in the community)
https://www.skool.com/k21academy/issues-qa-creating-task-definitions-configuring-services-and-cluster-using-fargate-post-here



Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

Error: "CannotPullContainerError: pull image manifest has been retried 7 times: failed to resolve ref docker.io/library/httpd:2.4"

This error indicates that ECS is unable to pull the Docker image (httpd:2.4) from Docker Hub. The issue could arise from multiple causes:



1. Network Connectivity Issues

Cause: The ECS cluster might not have proper network access to Docker Hub, possibly due to VPC, security groups, or firewall settings.

Resolution: Verify that the ECS cluster has internet access if pulling from Docker Hub (if using a private VPC, ensure a NAT Gateway or Internet Gateway is properly configured).

Double-check your ECS security group settings to ensure they allow outbound internet traffic.

2. Incorrect Image Name/Tag

Cause: The image name or tag (httpd:2.4) may be incorrect, or the image might not exist in Docker Hub.

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

Resolution: Confirm that the image exists in Docker Hub by trying to pull it manually:

docker pull httpd:2.4

If the image is not available or incorrectly referenced, update your ECS Task Definition to use the correct image name/tag.

3. Docker Hub Rate Limits

Cause: Docker Hub has rate limits that may be exceeded, particularly with anonymous pulls. This can result in a failure to pull the image.

Resolution: If you hit Docker Hub rate limits, either wait for them to reset or consider using Amazon ECR (Elastic Container Registry) as an alternative.

For public images, consider switching to Amazon ECR or using a private Docker registry to avoid rate limit issues.

4. Authentication Issues (Private Repositorie

Cause: If the image you're trying to pull is hosted in a private registry (Docker Hub, Amazon ECR), your ECS task definition might be missing the necessary authentication credentials.

Resolution: For Docker Hub: Ensure you have properly configured the ECS task definition to use Docker Hub credentials (username/password or access tokens).

For Amazon ECR: Ensure that your ECS cluster has proper IAM permissions to access the ECR repository and that you're using the correct image URI format.

5. Task Definition Misconfiguration

Cause: The ECS task definition might be incorrectly configured or referencing an invalid image repository.

Resolution: Double-check your ECS Task Definition:

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

Ensure the image URI is correct.

If using a private registry, ensure that the appropriate authentication is set up.

Update the task definition if necessary and redeploy your service.

6. Temporary Docker Hub Outage

Cause: There might be a temporary issue with Docker Hub itself, causing image pull failures.

Resolution: Check Docker Hub's status page or community forums to see if there's an ongoing outage.

If it's a temporary issue, retry pulling the image after some time.

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.

13 SUMMARY

This activity guide covers steps for:

- Create ECS Cluster
- Create Task Definition
- Create a Service
- Delete Resources

K21Academy

Copyright© K21Academy | All Rights Reserved
Sharing, Reselling, or duplication of this content is strictly prohibited without K21Academy's written permission.